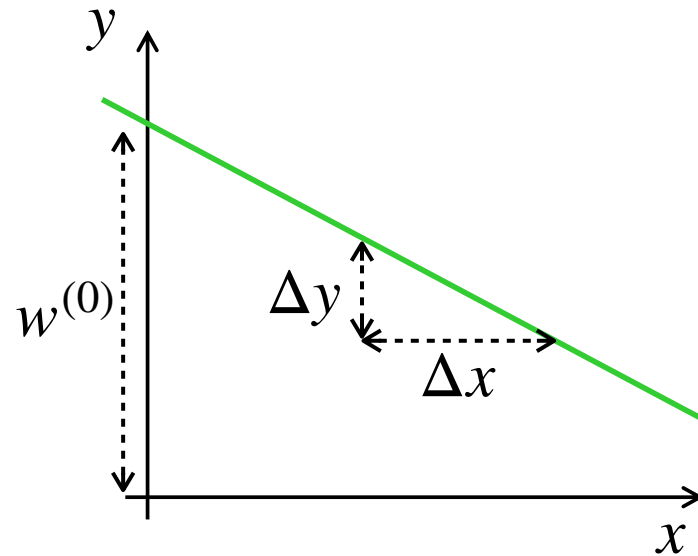


Lecture 3: Linear Models for Classification (Part 1)

Recap: A simple parametric model: The line



- Defined by 2 parameters
 - The y -intercept $w^{(0)}$
 - The slope $w^{(1)} = \frac{\Delta y}{\Delta x}$
- Mathematically, a line is expressed as

$$y = w^{(0)} + w^{(1)}x$$

Recap: Hyperplane

- This can be generalized to higher input dimensions
- In dimension D , we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \dots + w^{(D)}x^{(D)} = \mathbf{w}^T \begin{bmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{bmatrix}$$

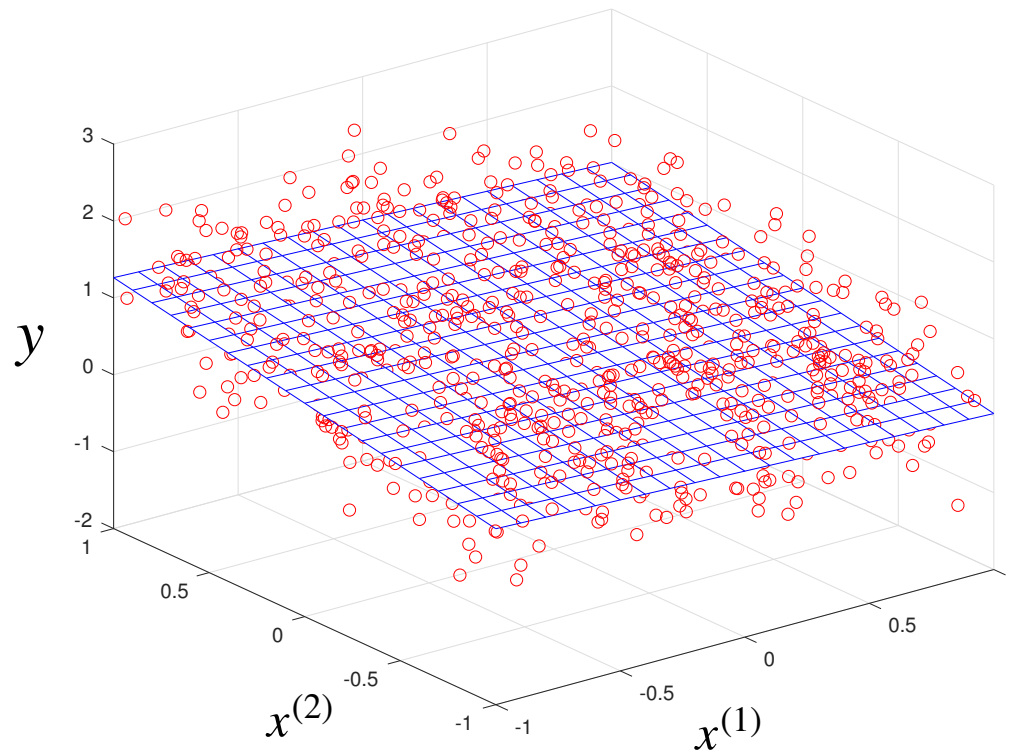
- Ultimately, whatever the input dimension, we can write

$$y = \mathbf{w}^T \mathbf{x}$$

with $\mathbf{x} \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$

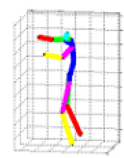
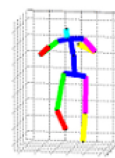
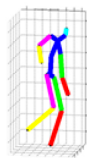
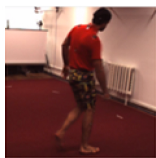
Recap: Linear regression

- Given N noisy pairs $\{(\mathbf{x}_i, y_i)\}$, where $\mathbf{x}_i \in \mathbb{R}^D$ (below, $D = 2$), find the parameters $\mathbf{w}^*$ that best fit these observations



Recap: Dealing with multiple output dimensions

- The previous examples assumed that the output was a single value, i.e., $y_i \in \mathbb{R}$
- In practice, however, one may want to output multiple values for a given input, i.e., $\mathbf{y}_i \in \mathbb{R}^C$, with $C > 1$
- E.g., human pose estimation
 - We aim to predict the 3D position of each joint, i.e., for J joints, $C = 3J$



Recap: Multi-output linear regression: Model

- To output multiple values, the linear model cannot just rely on a vector $\mathbf{w}$, because the product $\mathbf{w}^T \mathbf{x}_i$ yields a single value
- We therefore use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1) \times C}$, such that

$$\hat{\mathbf{y}}_i = \mathbf{W}^T \mathbf{x}_i = \begin{bmatrix} \mathbf{w}_{(1)}^T \\ \mathbf{w}_{(2)}^T \\ \vdots \\ \mathbf{w}_{(C)}^T \end{bmatrix} \mathbf{x}_i$$

where each $\mathbf{w}_{(j)}$ is a $(D + 1)$ -dimensional vector, used to predict one output dimension

Recap: Multi-output linear regression: Training

- Since the outputs are multi-dimensional, we need to slightly modify the loss function
- The multi-dimensional counterpart of the 1D least-square loss is the squared Euclidean distance
- So we can write multi-output linear regression as

$$\min_{\mathbf{W}} \sum_{i=1}^N \|\mathbf{W}^T \mathbf{x}_i - \mathbf{y}_i\|^2$$

- To solve this problem, we search for $\mathbf{W}^*$ such that the gradient becomes 0

Recap: Multi-output linear regression: Solution

- We can again group the inputs in a matrix $\mathbf{X} \in \mathbb{R}^{N \times (D+1)}$, but the outputs now need to be grouped in a matrix (not a vector anymore), as

$$\mathbf{Y} = \begin{bmatrix} \mathbf{y}_1^T \\ \mathbf{y}_2^T \\ \vdots \\ \mathbf{y}_N^T \end{bmatrix} = \begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(C)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(C)} \\ \vdots & \vdots & \vdots & \vdots \\ y_N^{(1)} & y_N^{(2)} & \cdots & y_N^{(C)} \end{bmatrix} \in \mathbb{R}^{N \times C}$$

- Then, following the same strategy as before, we have

$$\mathbf{W}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y} = \mathbf{X}^\dagger \mathbf{Y}$$

Goals of today's lecture

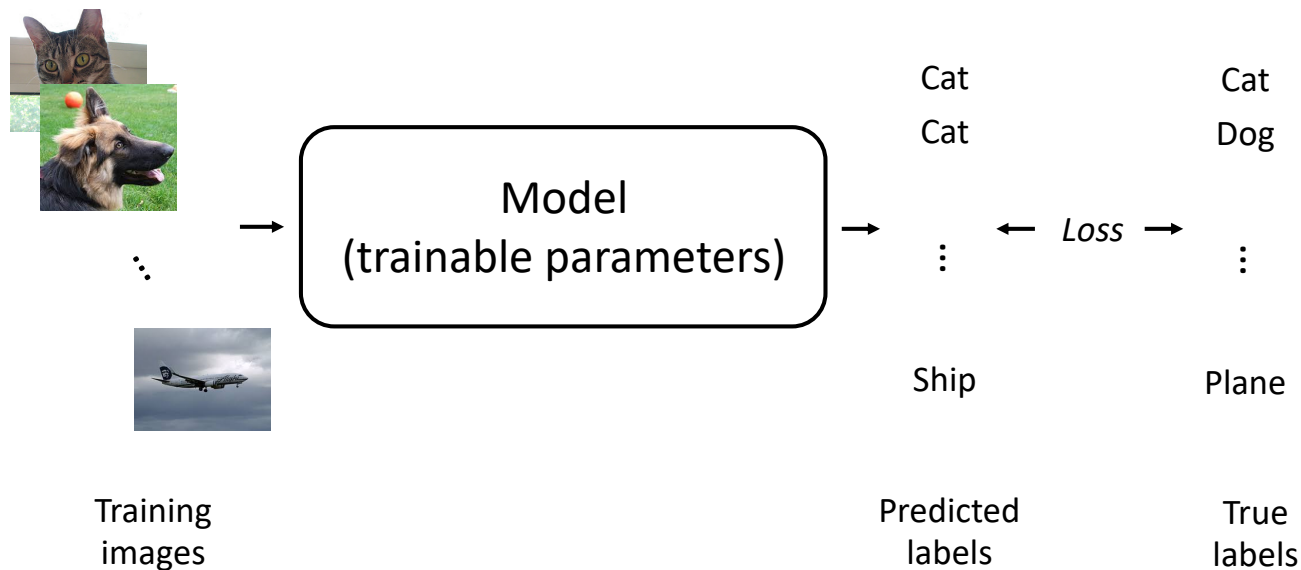
- Introduce the notion of loss function and empirical risk
- Move from regression to classification
- Introduce the formulation for logistic regression
- Introduce basic minimization concepts

Interlude

Loss Function and Empirical Risk
(These are general ML concepts)

Loss function

- The loss function $\ell(\hat{y}_i, y_i)$ computes an error value between the prediction and the true value
 - This is a general ML concept, not only for linear regression
 - We will see different loss functions in the upcoming lectures



Loss function: Examples

- So far, we have seen the following loss functions for linear regression
 - Single output dimension: $\ell(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$
 - Multiple output dimensions: $\ell(\hat{\mathbf{y}}_i, \mathbf{y}_i) = \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2$
- In principle, other loss functions could be used for regression
 - For example, using the MAE evaluation metric: $\ell(\hat{y}_i, y_i) = |\hat{y}_i - y_i|$
- As we will see later today and in future lectures, other loss functions can be used for classification

Empirical risk

- Given N training samples $\{(\mathbf{x}_i, y_i)\}$, the empirical risk is defined as

$$R(\{\mathbf{x}_i\}, \{y_i\}, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

where $\{\mathbf{x}_i\}$ and $\{y_i\}$ are the sets of training inputs and labels, respectively

- During training, our goal is to find the parameters $\mathbf{w}$ (e.g., $w^{(0)}$ and $w^{(1)}$) that minimize the empirical risk
 - Note that the risk depends on $\mathbf{w}$ via $\hat{y}_i$

Minimizing the risk

- This is expressed as the optimization problem

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

- We can also write that the best parameters are the solution

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

- This is what we have done when training our linear regressor
 - Our least-square minimization problem is one example of empirical risk minimization, using the square loss

End of the interlude

Let's now move to linear classification

Recap: From regression to classification

- Predict one discrete label for a given sample
 - E.g., binary classification for movies (like vs not like)

5	Column1	Column2
6	A very, very, very slow-moving, aimless movie about a distressed, drifting young man.	0
8	Not sure who was more lost - the flat characters or the audience, nearly half of whom walked out.	0
10	Attempting artiness with black & white and clever camera angles, the movie disappointed - became e	0
11	Very little music or anything to speak of.	0
13	The best scene in the movie was when Gerardo is trying to find a song that keeps running through his	1
15	The rest of the movie lacks art, charm, meaning... If it's about emptiness, it works I guess because it's	0
16	Wasted two hours.	0
18	Saw the movie today and thought it was a good effort, good messages for kids.	1
20	A bit predictable.	0
22	Loved the casting of Jimmy Buffet as the science teacher.	1
23	And those baby owls were adorable.	1
25	The movie showed a lot of Florida at it's best, made it look very appealing.	1
26	The Songs Were The Best And The Muppets Were So Hilarious.	1

- E.g., multi-class image recognition



Cat



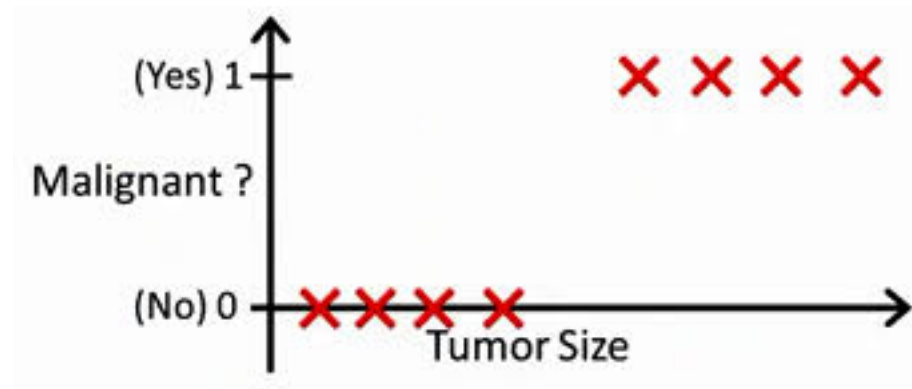
Dog



Car

Recap: Binary classification as regression

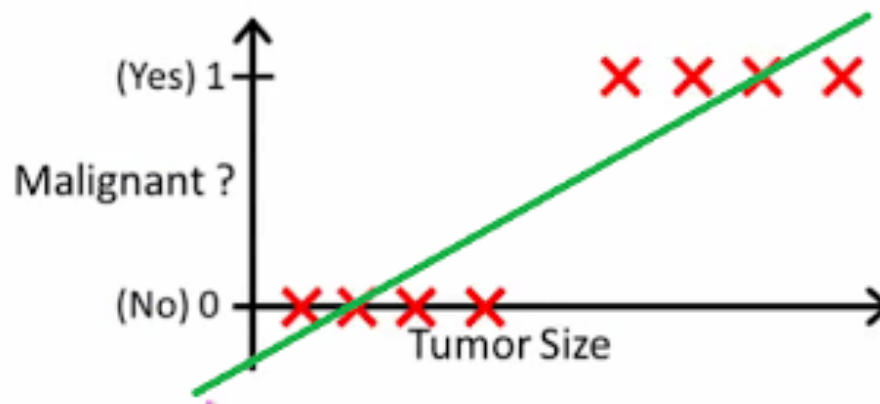
- Example (from Andrew Ng's course): Classify a tumor as benign vs malignant
 - 1D input (tumor size), 1D output (malignant vs benign)



Recap: Binary classification as regression

- Example (from Andrew Ng's course): Classify a tumor as benign vs malignant
 - 1D input (tumor size), 1D output (malignant vs benign)
 - Using a linear model, we can predict the label as

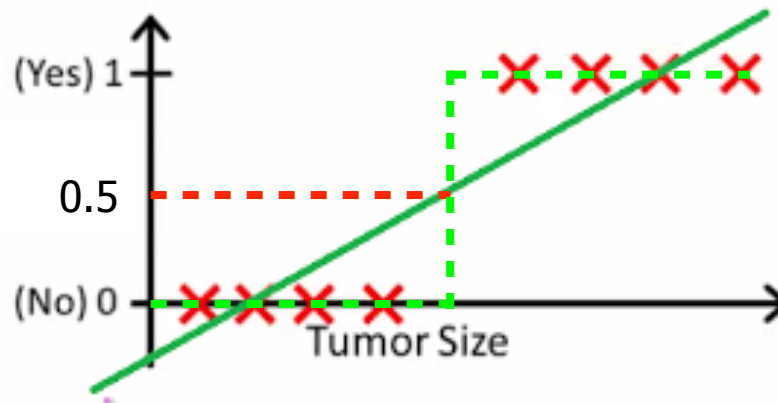
$$\hat{y} = w^{(0)} + w^{(1)}x$$



Recap: Least-square binary classification

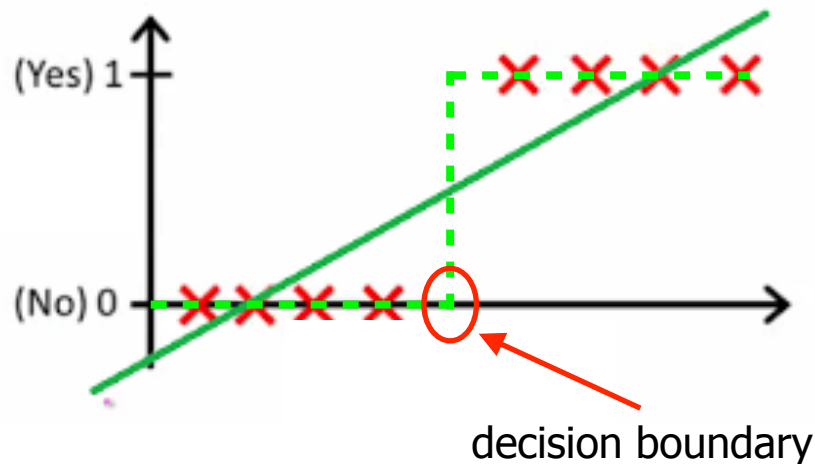
- Linear regression outputs a continuous value
 - We would rather like to have a discrete label, e.g., 1 or 0
 - This can be achieved by thresholding:

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y}_t \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \text{with } \hat{y}_t = \mathbf{x}_t^T \mathbf{w}^*$$



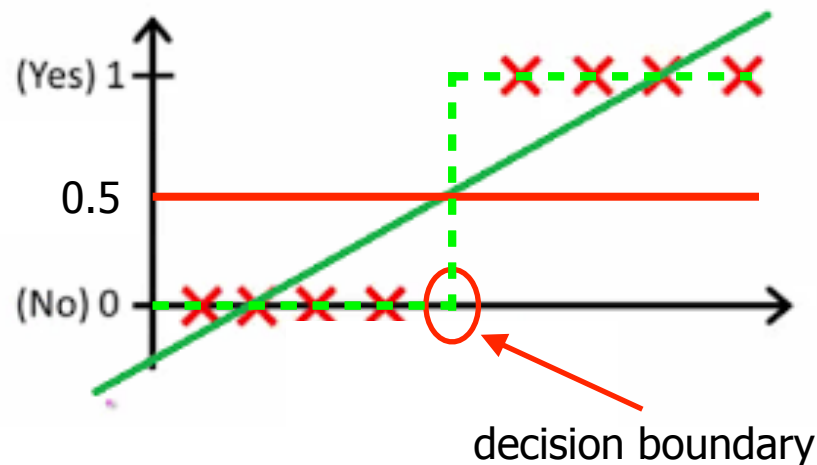
Decision boundary

- The point where the predicted label changes from 0 to 1 forms a *decision boundary*
 - With 1D inputs, it is a single point



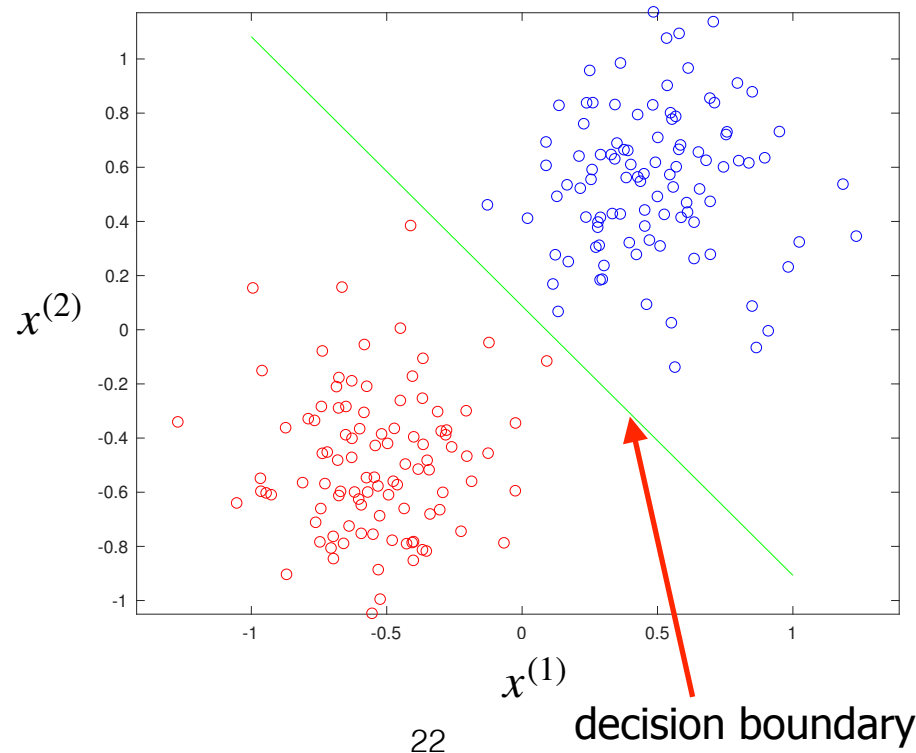
Decision boundary

- The point where the predicted label changes from 0 to 1 forms a *decision boundary*
 - With 1D inputs, it is a single point
 - It corresponds to the value in the input dimension where the linear model (green line) intersects the line $y = 0.5$ (red line)



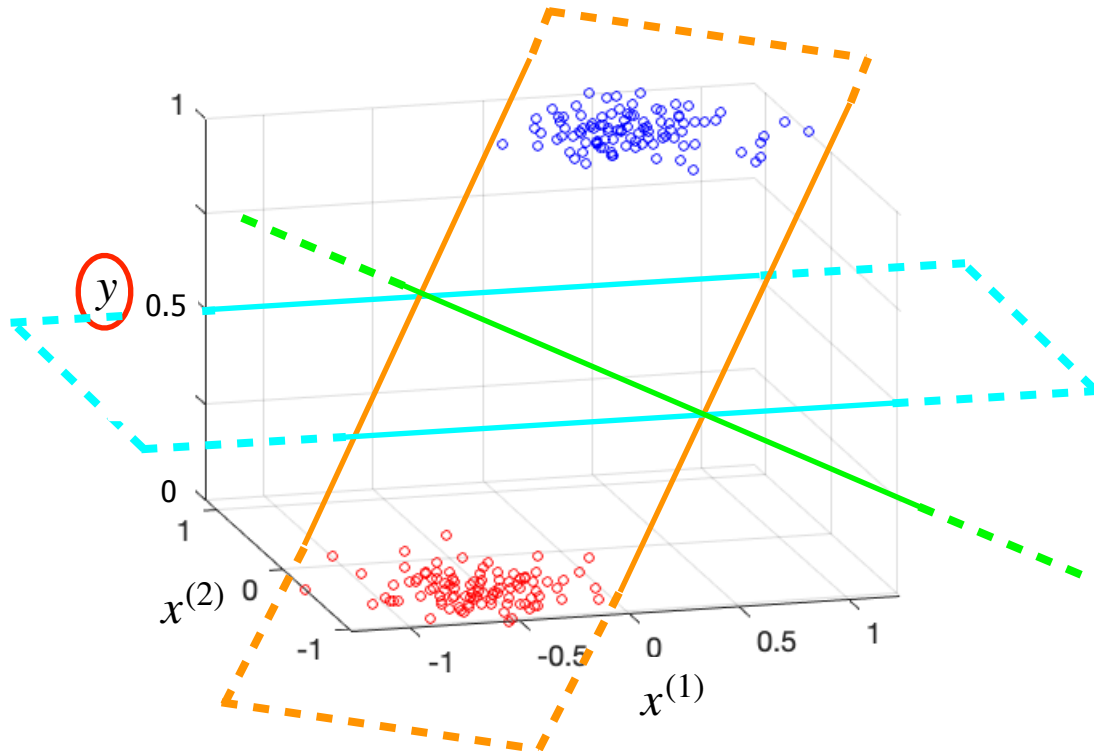
Decision boundary

- The point where the predicted label changes from 0 to 1 forms a *decision boundary*
 - With 2D inputs, it is a line



Decision boundary

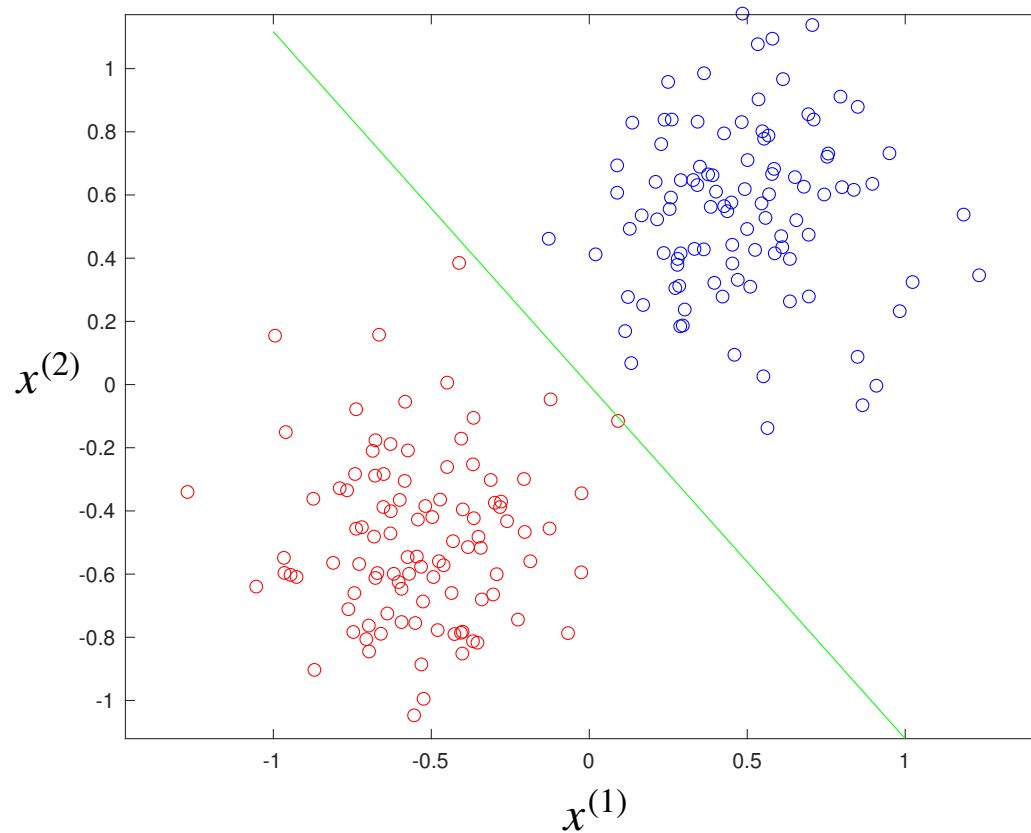
- The point where the predicted label changes from 0 to 1 forms a *decision boundary*
 - With 2D inputs, it is a line
 - To understand this, let's visualize the data in 3D (with y as 3rd dimension)



- The linear model encodes the plane that passes through the samples (orange plane)
- The decision boundary (green line) is the intersection of this plane with the plane $y = 0.5$ (cyan plane)

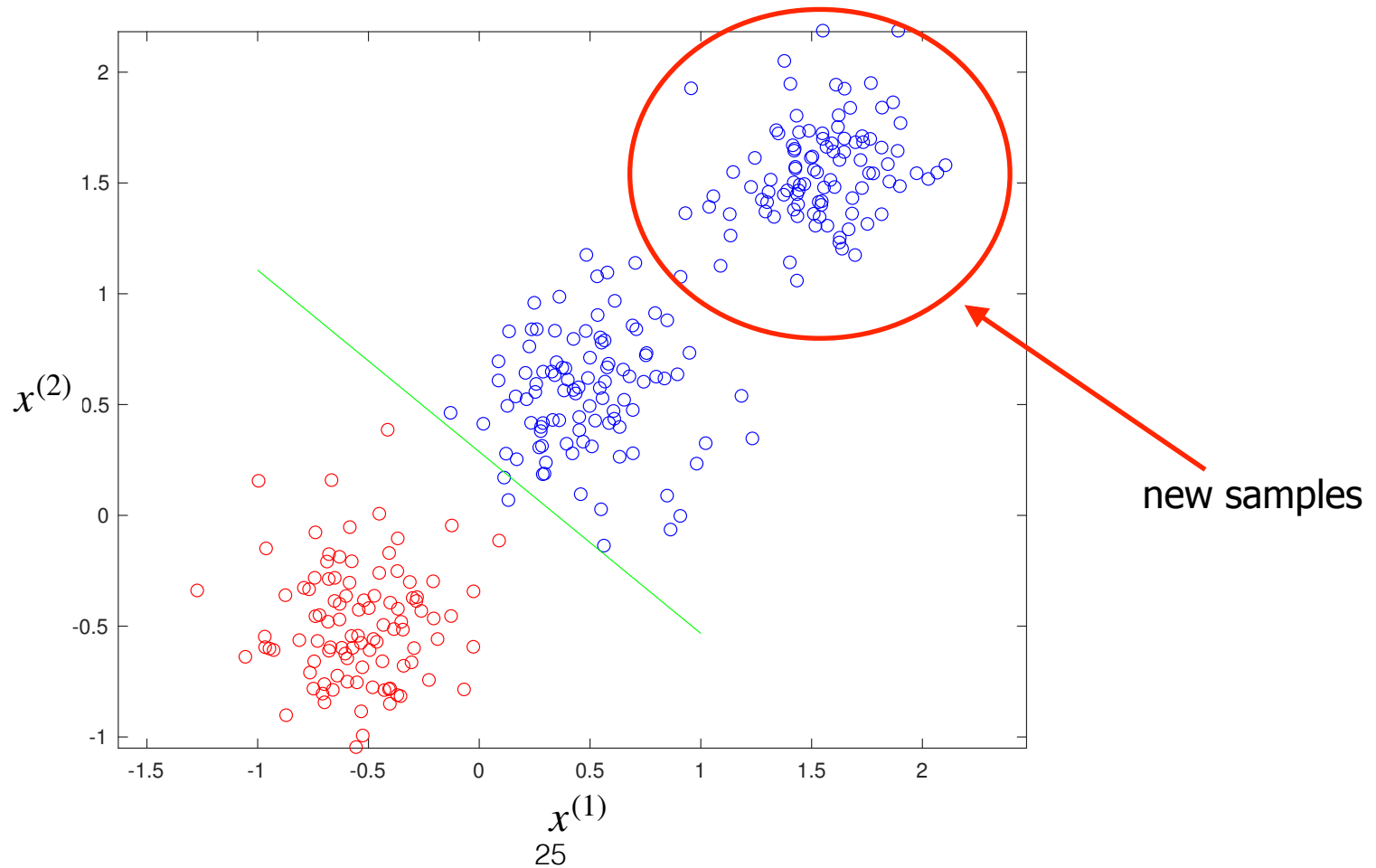
Recap: Least-square binary classification: Example

- 2D inputs, 2 classes
 - All training samples are correctly classified



Recap: Least-square binary classification: Failure

- However, adding new samples far from the decision boundary makes some of the training samples be misclassified

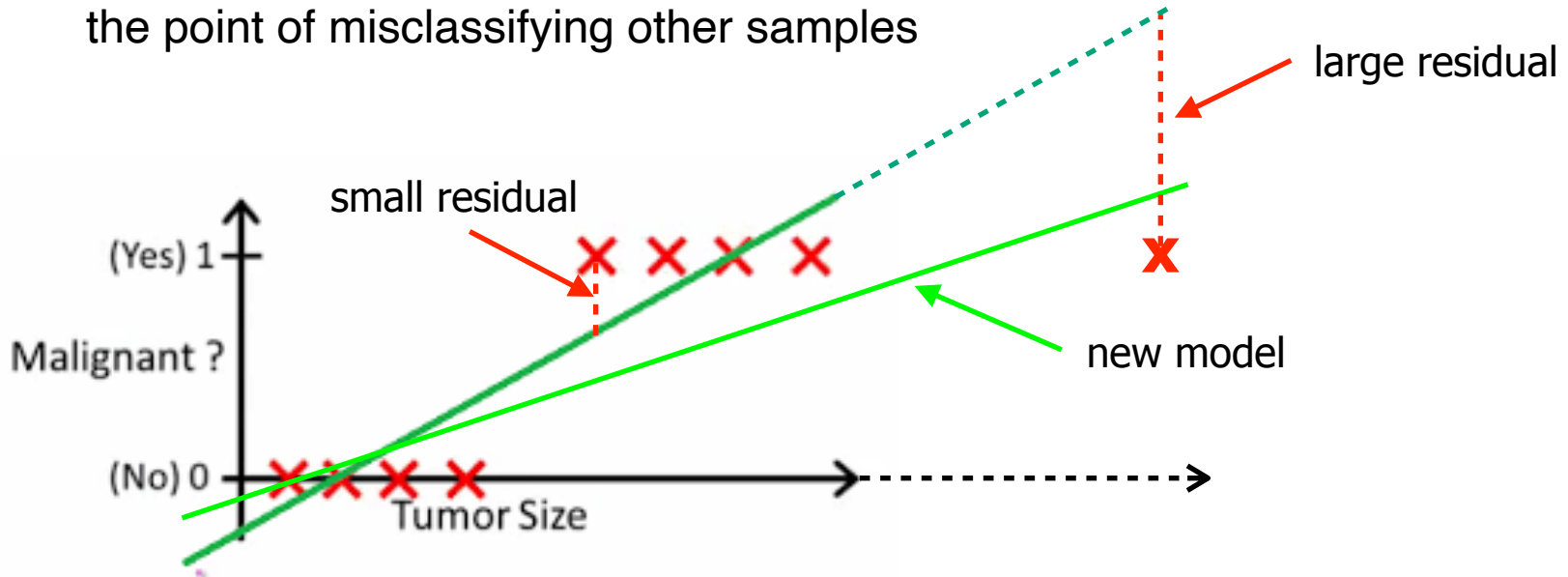


Failure: Geometric interpretation

- <https://observablehq.com/@yizhe-ang/interactive-visualization-of-linear-regression>

Failure: Geometric interpretation

- Back to the tumor prediction example
 - The residuals for “normal” samples are quite small
 - However, a sample with a much larger tumor size (with true label 1) would have a much larger residual
 - Training with such a sample would strongly affect the model parameters, to the point of misclassifying other samples



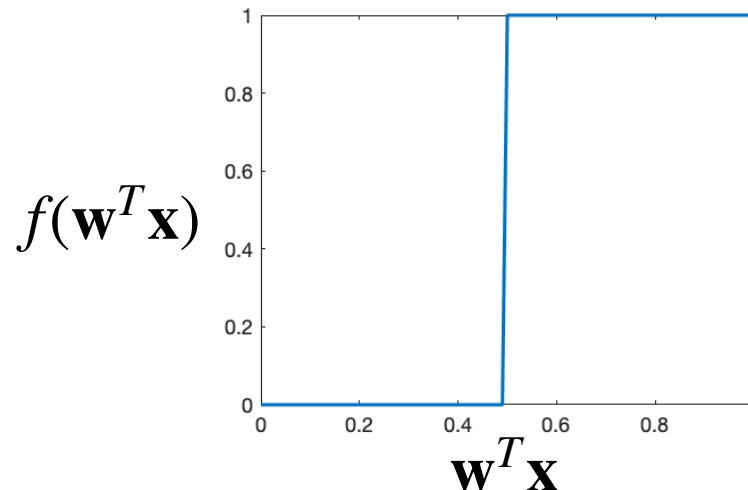
- This is because least-square classification fits a line to non-linear data

Adding non-linearity

- The output of the linear model $\mathbf{w}^T \mathbf{x}$ is a continuous value
- What we would really like is a discrete output, e.g.,

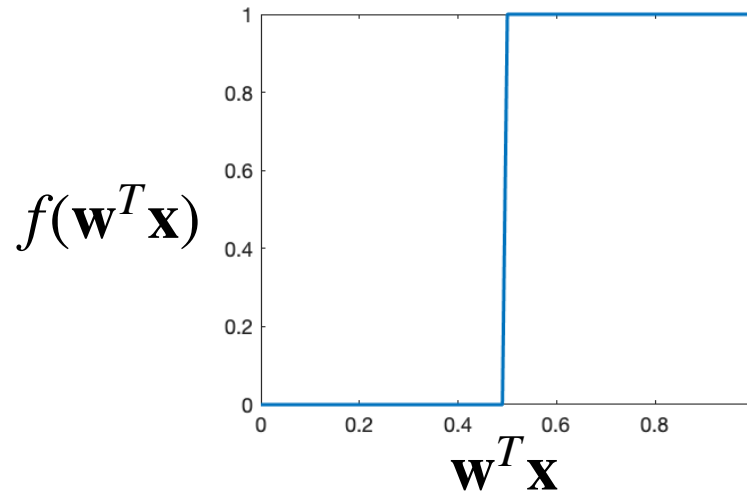
$$y = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

- This can be achieved by passing the output of the linear model through a step function



Step function: Drawback

- The gradient of the step function is not continuous
 - It is zero almost everywhere, except at $\mathbf{w}^T \mathbf{x} = 0.5$

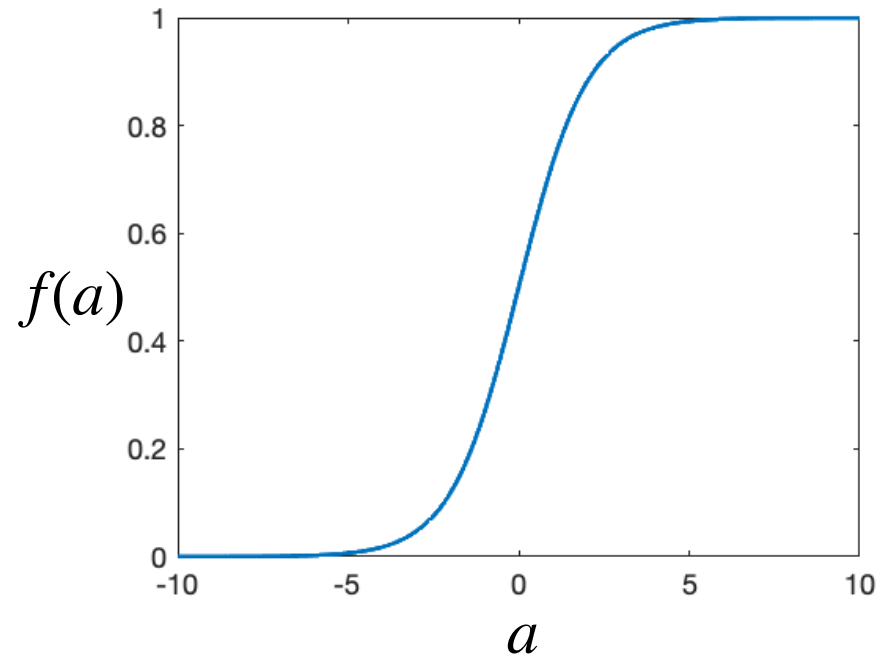


- This behavior typically makes optimization/training difficult
 - The Perceptron algorithm can nonetheless handle the 2-class case, but we will not cover it in this course

Adding non-linearity

- Instead, one can use a smooth approximation of the step function, such as the *logistic sigmoid function*

$$f(a) = \frac{1}{1 + \exp(-a)}$$

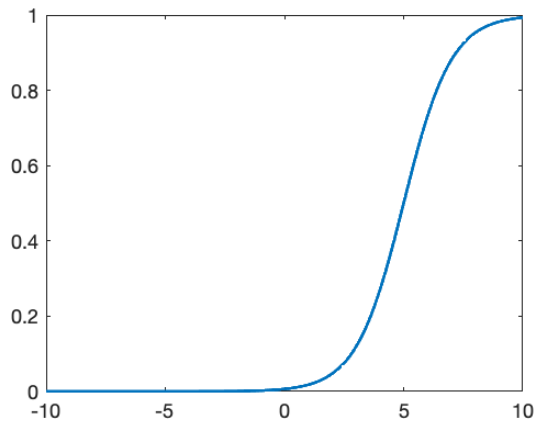


Logistic function: Effect of w

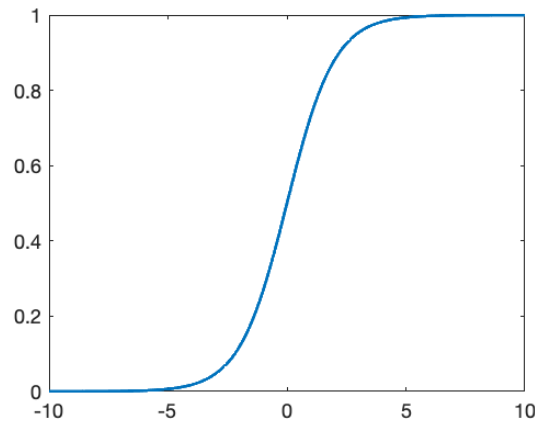
- With a 1D input x , applying the logistic function to the output of a linear model gives a prediction

$$\hat{y} = \frac{1}{1 + \exp(-w^{(0)} - w^{(1)}x)}$$

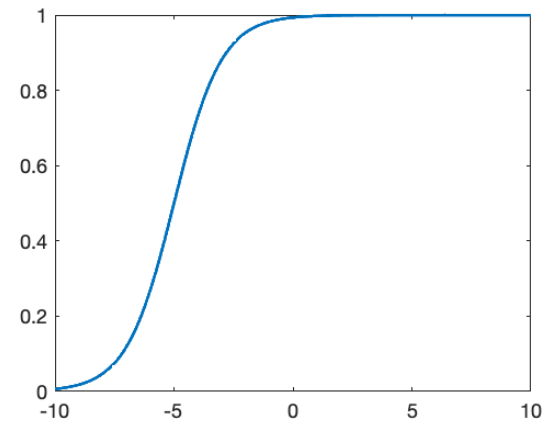
- Effect of $w^{(0)}$



$$w^{(0)} = -5, w^{(1)} = 1$$



$$w^{(0)} = 0, w^{(1)} = 1$$



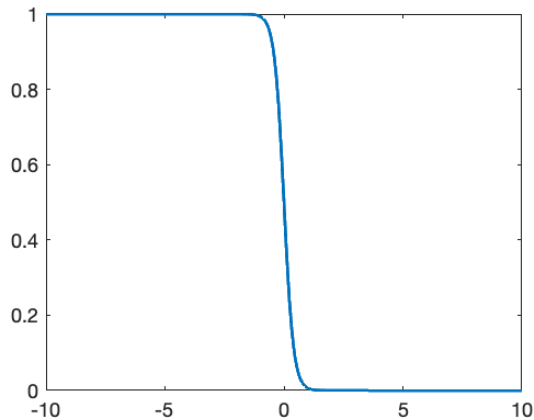
$$w^{(0)} = 5, w^{(1)} = 1$$

Logistic function: Effect of w

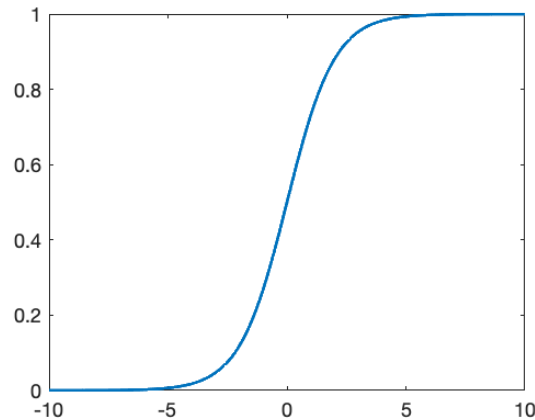
- With a 1D input x , applying the logistic function to the output of a linear model gives a prediction

$$\hat{y} = \frac{1}{1 + \exp(-w^{(0)} - w^{(1)}x)}$$

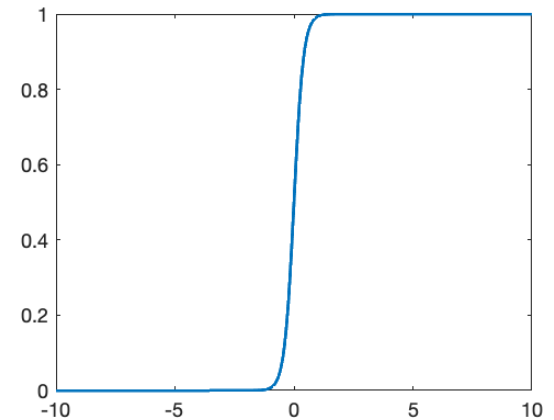
- Effect of $w^{(1)}$



$$w^{(0)} = 0, w^{(1)} = -5$$



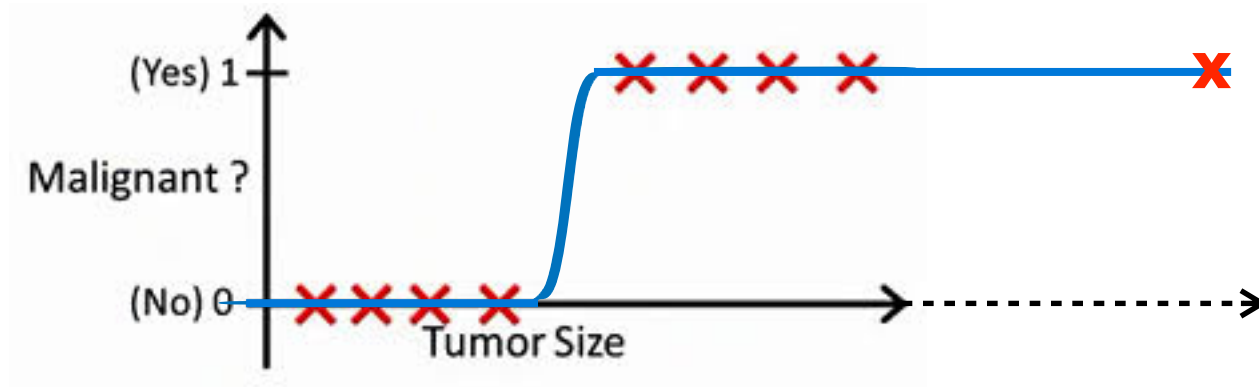
$$w^{(0)} = 0, w^{(1)} = 1$$



$$w^{(0)} = 0, w^{(1)} = 5$$

Fitting the tumor data

- Applying a logistic function to the output of the linear model can fit the data much better
- Even in the presence of outliers



Probabilistic interpretation

- In the binary case, with D dimensional inputs, we can write the prediction of the model as

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

- Such a prediction can be interpreted as the probability that $\mathbf{x}$ belongs to the positive class (or as a score for the positive class)
- The probability to belong to the negative class (or score for the negative class) is then computed as $1 - \hat{y}$

Logistic regression

- The model whose prediction is defined as

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

is called *logistic regression*

- Note that this name is slightly misleading, as this is truly a classification model, not a regression one
- At its heart, logistic regression still makes use of the linear model, but followed by a nonlinearity to better encode the underlying binary classification problem

Logistic regression: Training

- Given N training samples, to learn the parameters $\mathbf{w}$ of the logistic regression model, we need to define a loss function
- Instead of using the L^2 loss as in the least-square case, this is achieved by making use of the probabilistic interpretation
 - Because knowledge of probabilities is not required for this course, the derivation of the loss function that follows will be a bit hand-wavy.
 - Nevertheless, this loss remains intuitive: The loss is such that it aims to find the parameters that maximize the probability (or score) of the correct class for each training sample

Logistic regression: Training

- Specifically, the logistic regression loss is derived by first writing the following likelihood (think of it as a value representing how well the true labels are predicted given $\mathbf{w}$)

$$p(\mathbf{y} | \mathbf{w}) = \prod_{i=1}^N \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{1-y_i}$$

- Because a product of terms between 0 and 1 can be unstable to optimize, we take the logarithm of this likelihood, and take its negative to define an empirical risk to minimize, written as

$$R(\mathbf{w}) = - \sum_{i=1}^N (y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i))$$

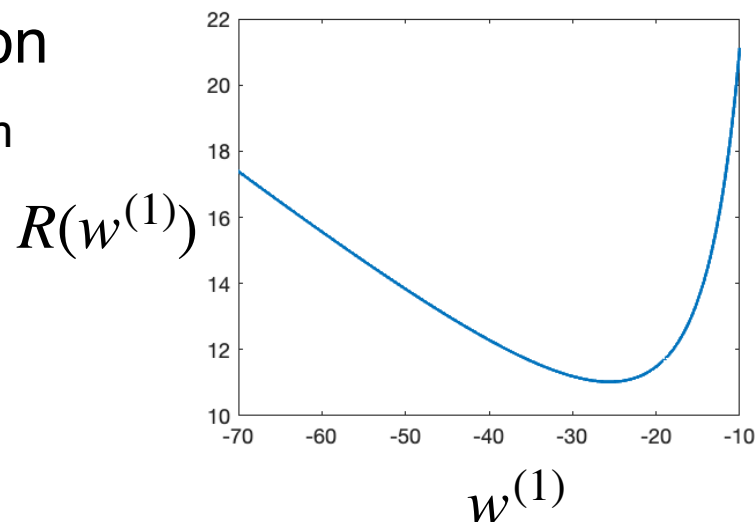
- This loss function is known as the *cross-entropy*

Logistic regression: Training

- Since the true y_i is either 0 or 1, the cross-entropy can be re-written as

$$R(\mathbf{w}) = - \sum_{i \in \text{positive samples}} \ln(\hat{y}_i) - \sum_{i \in \text{negative samples}} \ln(1 - \hat{y}_i)$$

- The cross-entropy is a convex function
 - However, its minimization has no closed-form solution
- We optimize it via gradient descent
 - Let us now review this algorithm

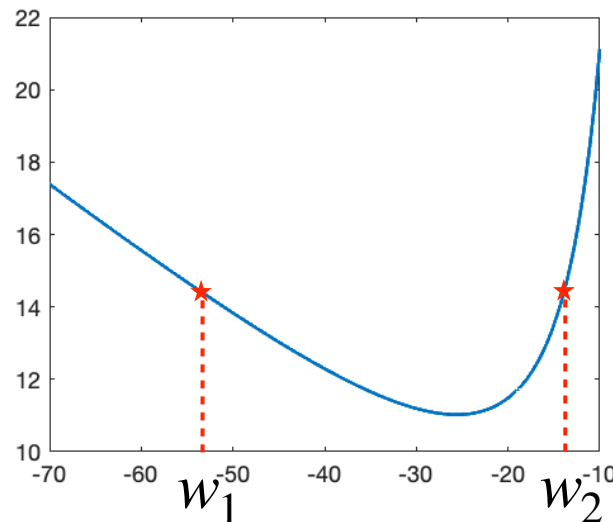


Iterative, Gradient-based Optimization

Minimizing a function

- With 1D functions for which the solution cannot be computed in closed form, one can nonetheless use the derivative
 - Recall that it represents the slope at a given point, and thus indicates in which direction to move to reach a lower function value

At w_1 , the slope is negative. However, one should move in the positive direction ($\Delta w > 0$) to go towards the minimum



At w_2 , the slope is positive. However, one should move in the negative direction ($\Delta w < 0$) to go towards the minimum

- This indicates that one should move in the direction opposite to the derivative for minimization

Minimizing a function

- Algorithm:

1. Initialize w_0 (e.g., randomly)

2. While not converged

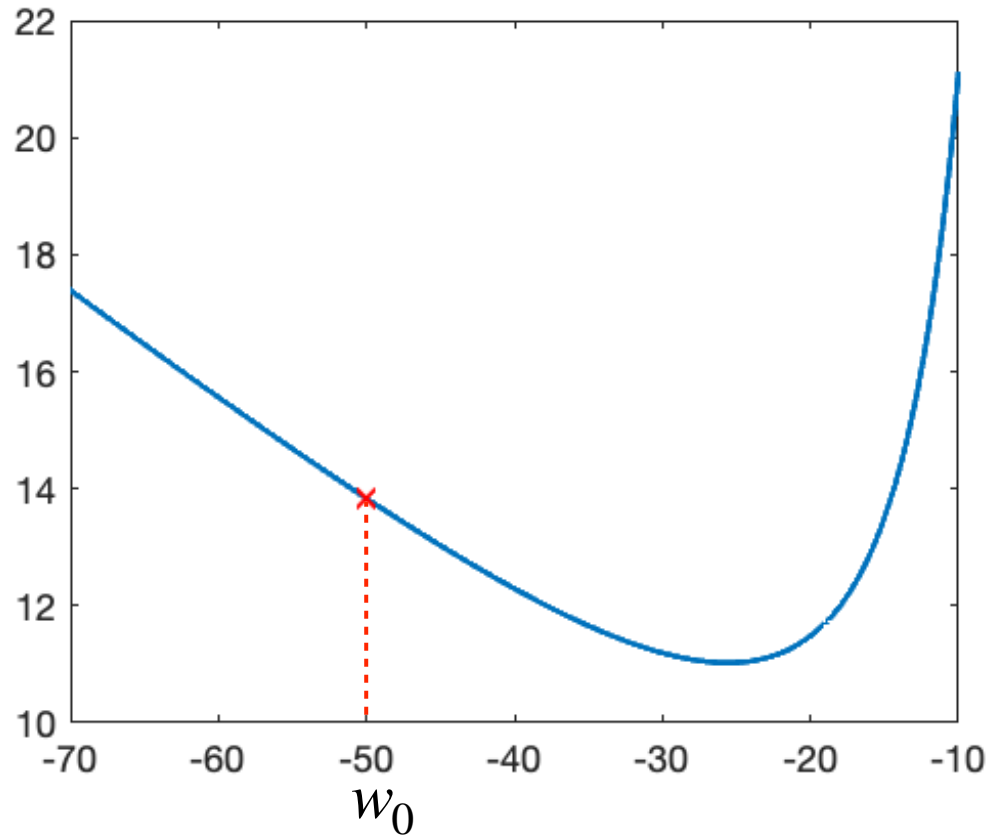
- 2.1. Update $w_k \leftarrow w_{k-1} - \eta \frac{dR(w_{k-1})}{dw}$

- η defines the step size of each iteration

- In ML, it is often referred to as *learning rate*

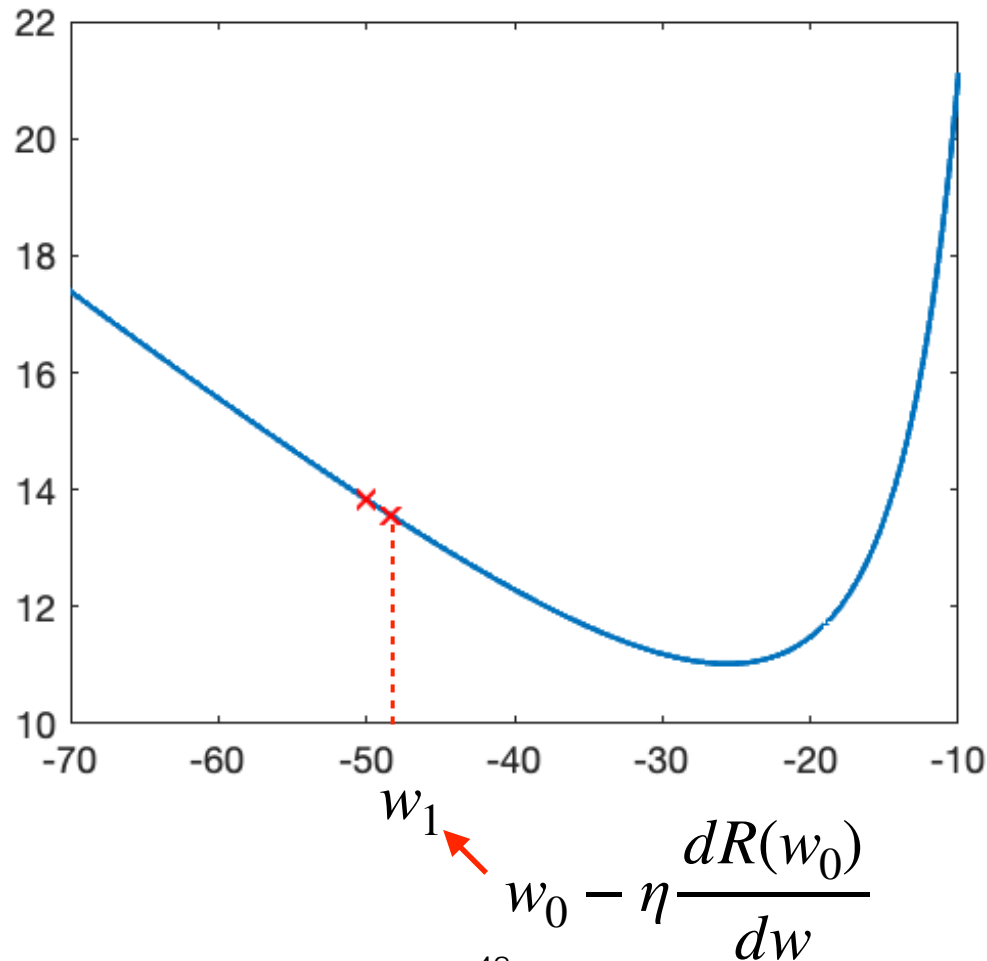
Minimizing a convex function

- Example:



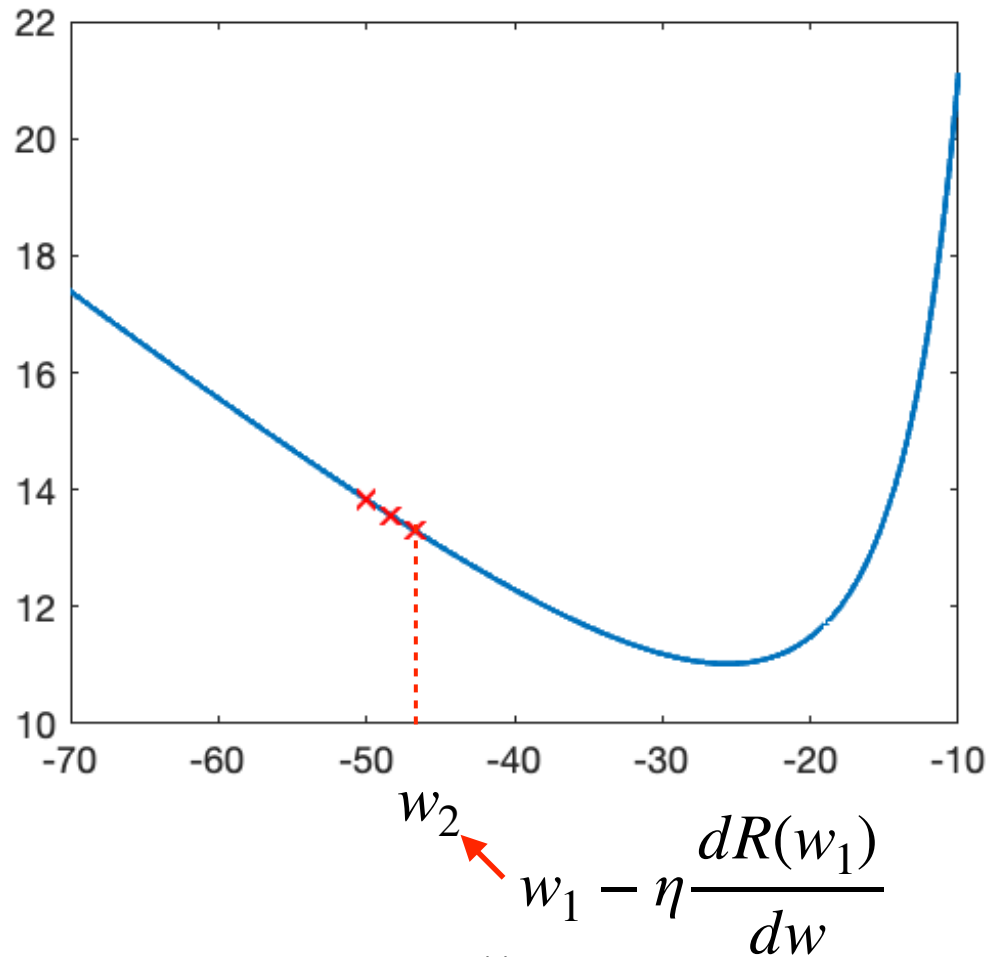
Minimizing a convex function

- Example:



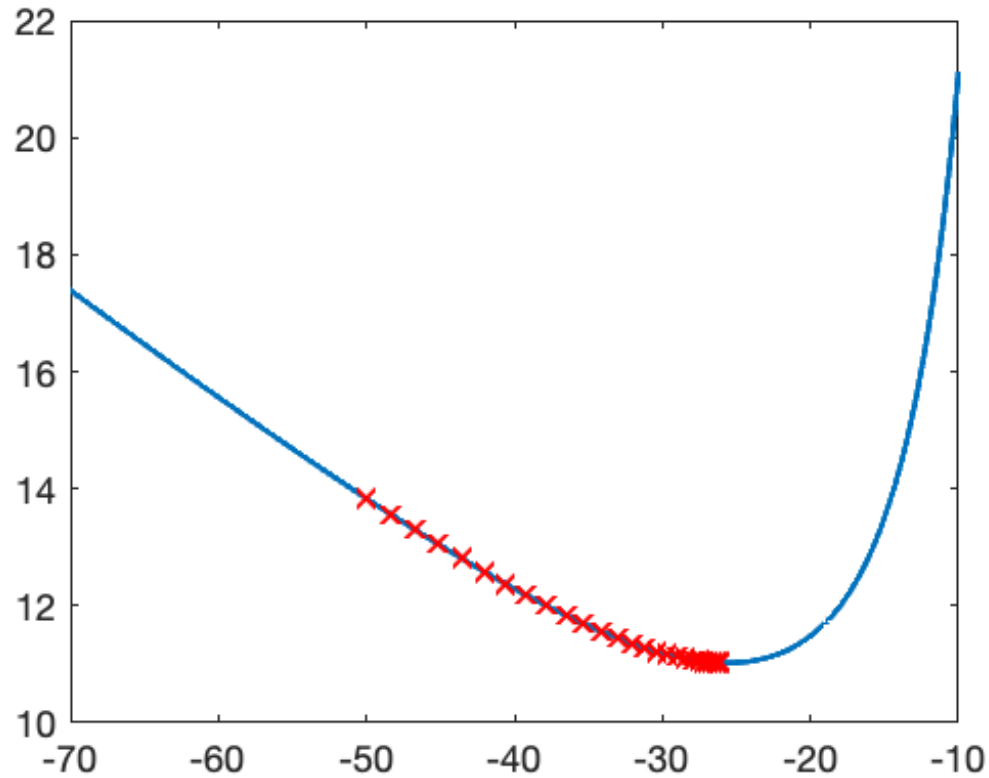
Minimizing a convex function

- Example:



Minimizing a convex function

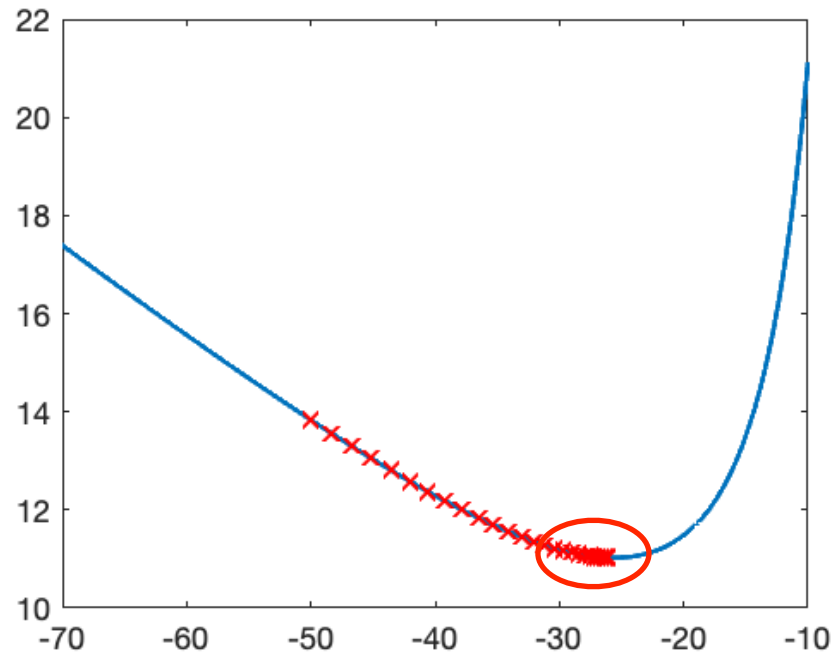
- Example:



Minimizing a convex function

- Convergence:

- Because the derivative becomes smaller $\left(\frac{dR(w)}{dw} \rightarrow 0 \right)$, the update Δw becomes smaller, and the algorithm converges



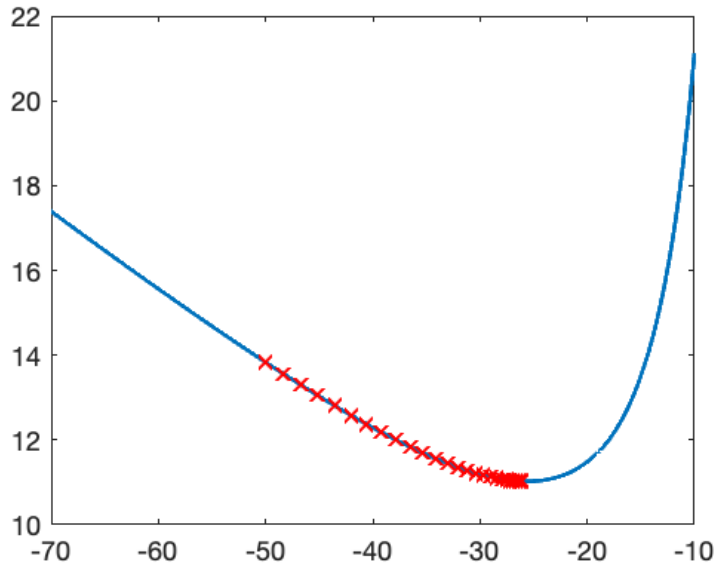
Minimizing a function

- Potential convergence criteria:
 - Change in function values less than threshold: $|R(w_{k-1}) - R(w_k)| < \delta_R$
 - Change in parameter value less than threshold: $|w_{k-1} - w_k| < \delta_w$
 - Maximum number of iterations reached
 - No guarantee to have reached the minimum

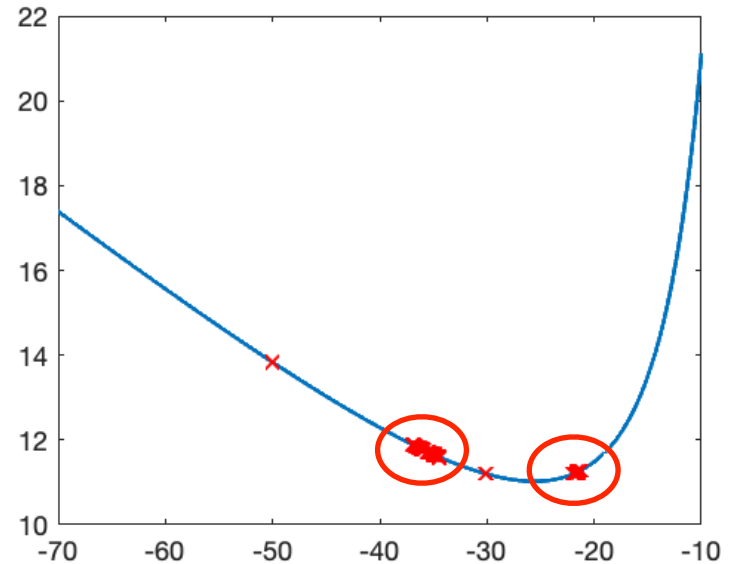
Minimizing a convex function

- Influence of η :

$$\eta = 10$$



$$\eta = 120$$

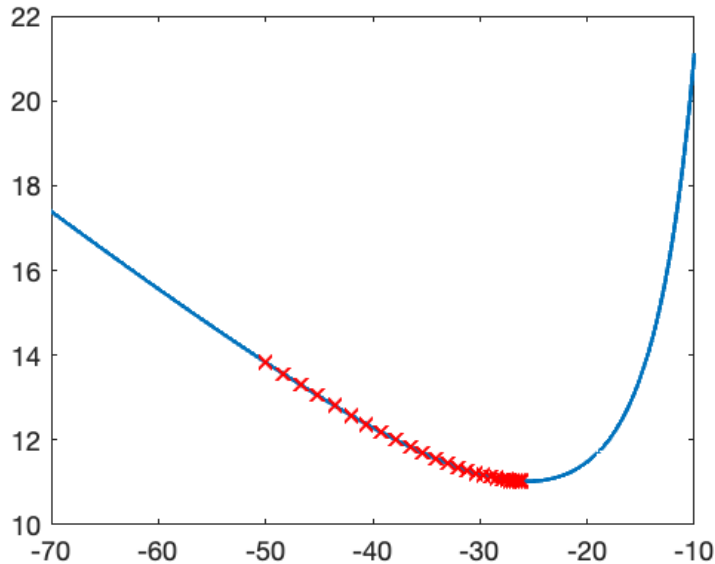


The steps are too large and the algorithm starts jumping between these two points

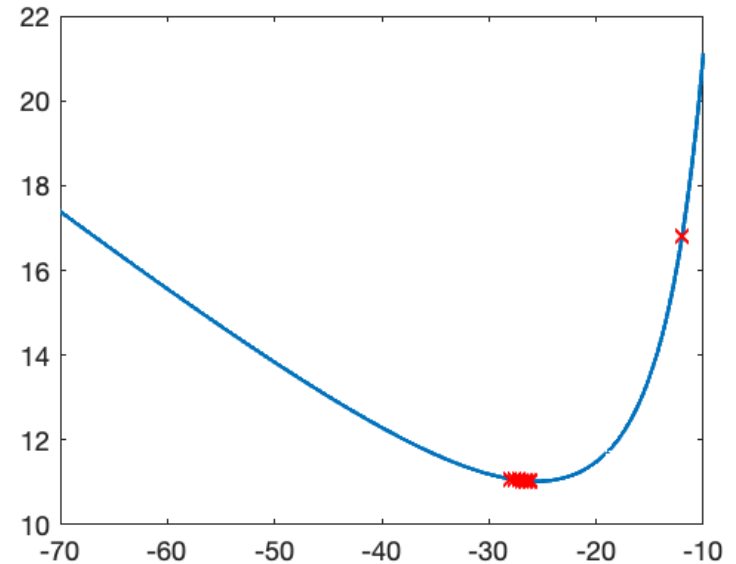
Minimizing a convex function

- Influence of the initial w_0 :

$$w_0 = -50$$



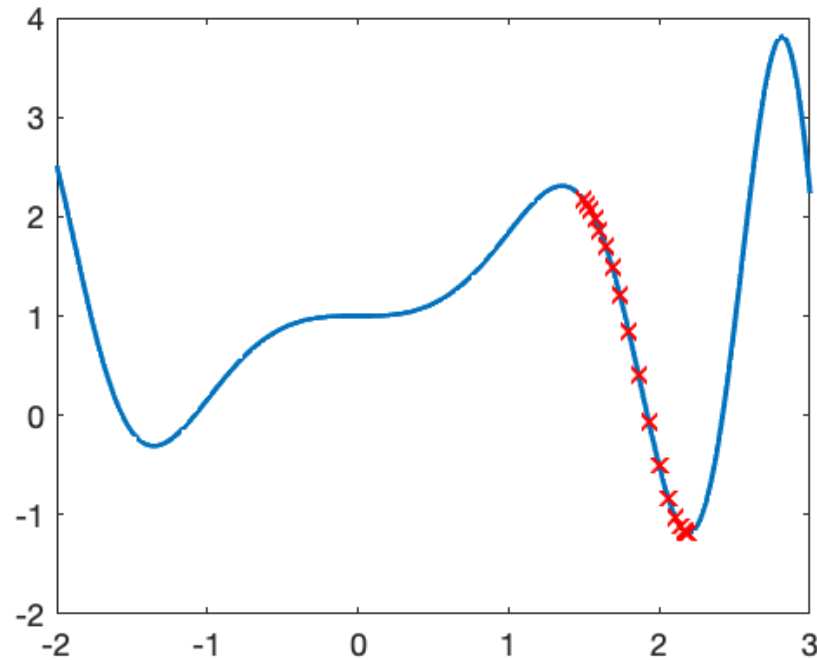
$$w_0 = -12$$



Still converges to the same point, but faster

Minimizing a non-convex function

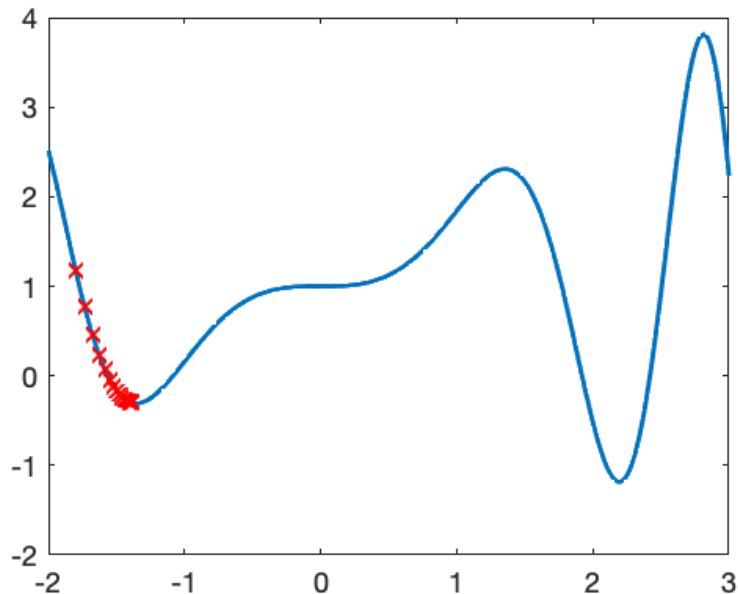
- The same iterative algorithm as before can be used



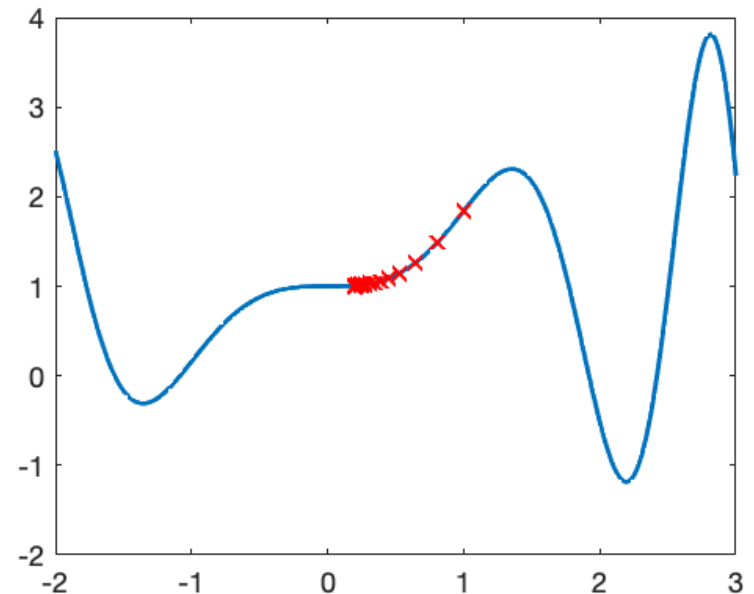
Minimizing a non-convex function

- However, different initializations yield different results
 - Before, $w_0 = 1.5$ and optimization reached the global minimum

$w_0 = -1.8$
Local minimum



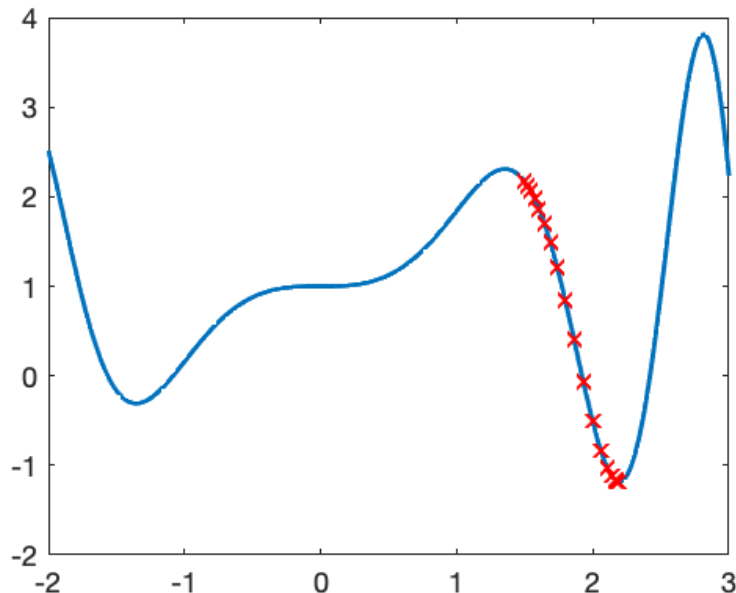
$w_0 = 1$
Saddle point



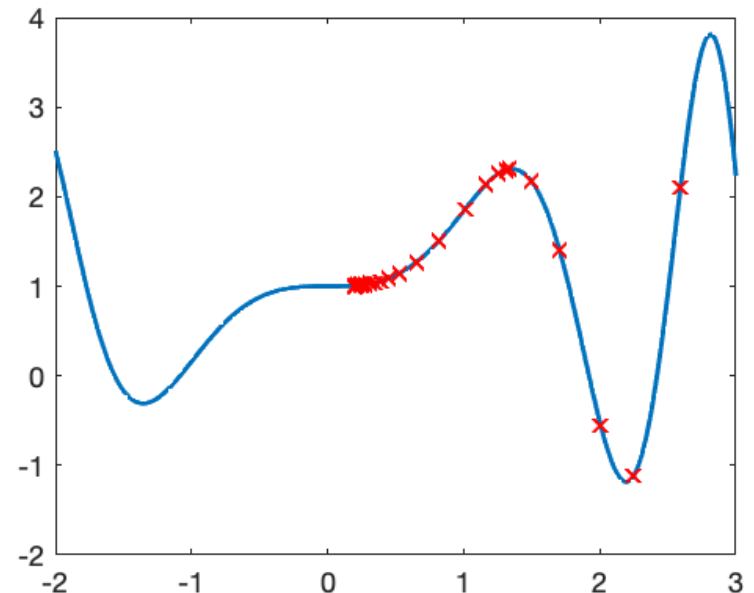
Minimizing a non-convex function

- And the results remain sensitive to the learning rate η
 - Note that η is problem-dependent and the values used here are very different from those used in the convex example before

$\eta = 0.01$, $w_0 = 1.5$
Global minimum



$\eta = 0.1$, $w_0 = 1.5$
Saddle point



Recap: Gradient

- In practice, we have more than one parameter, i.e., $\mathbf{w} \in \mathbb{R}^{D+1}$, and we thus use the function gradient:

$$\nabla_{\mathbf{w}} R(\mathbf{w}) = \begin{bmatrix} \frac{\partial R}{\partial w^{(0)}} \\ \frac{\partial R}{\partial w^{(1)}} \\ \frac{\partial R}{\partial w^{(2)}} \\ \vdots \\ \frac{\partial R}{\partial w^{(D)}} \end{bmatrix} \in \mathbb{R}^D$$

where $\frac{\partial R}{\partial w^{(j)}}$ denotes the partial derivative w.r.t. variable $w^{(j)}$

Recap: Properties of gradient

- The gradient at a point $\mathbf{w}$ has the direction of greatest increase of the function at $\mathbf{w}$
- Its magnitude is the rate of increase in that direction

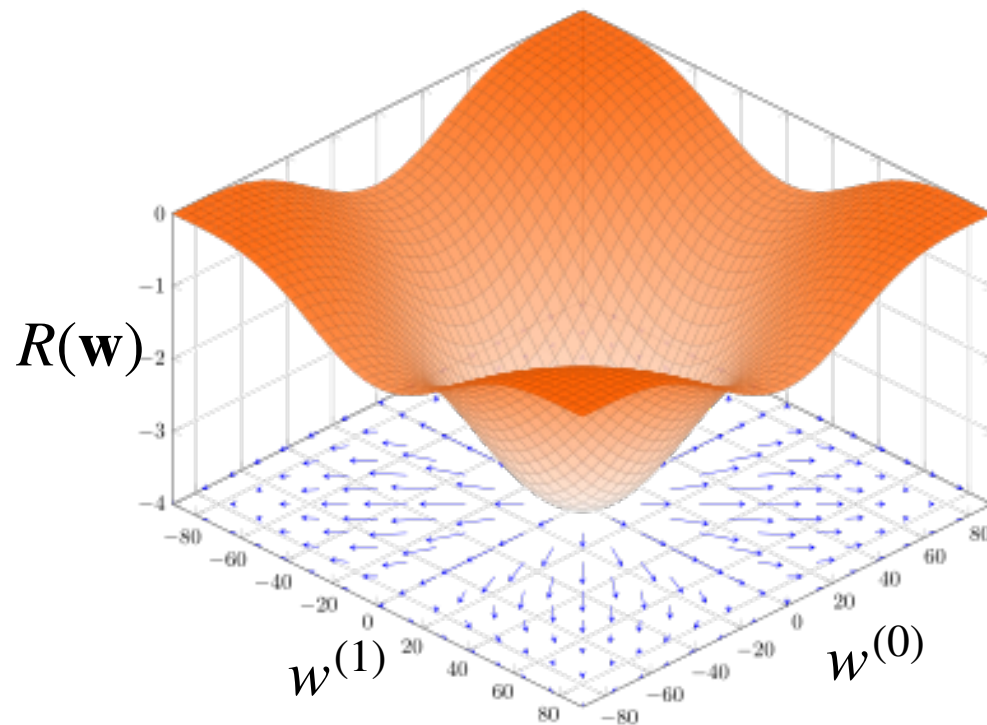


Figure from Wikipedia

Minimizing multi-variate functions

- As in the 1D case, to minimize a function that has no closed-form solution, we can exploit the fact that the gradient points in the direction of maximum function increase
- To decrease the function, we therefore want to move in the direction opposite to the gradient
- This yields an iterative algorithm that generalizes the one-variable one we saw before to multiple variables
 - This is called *gradient descent* (or steepest descent)

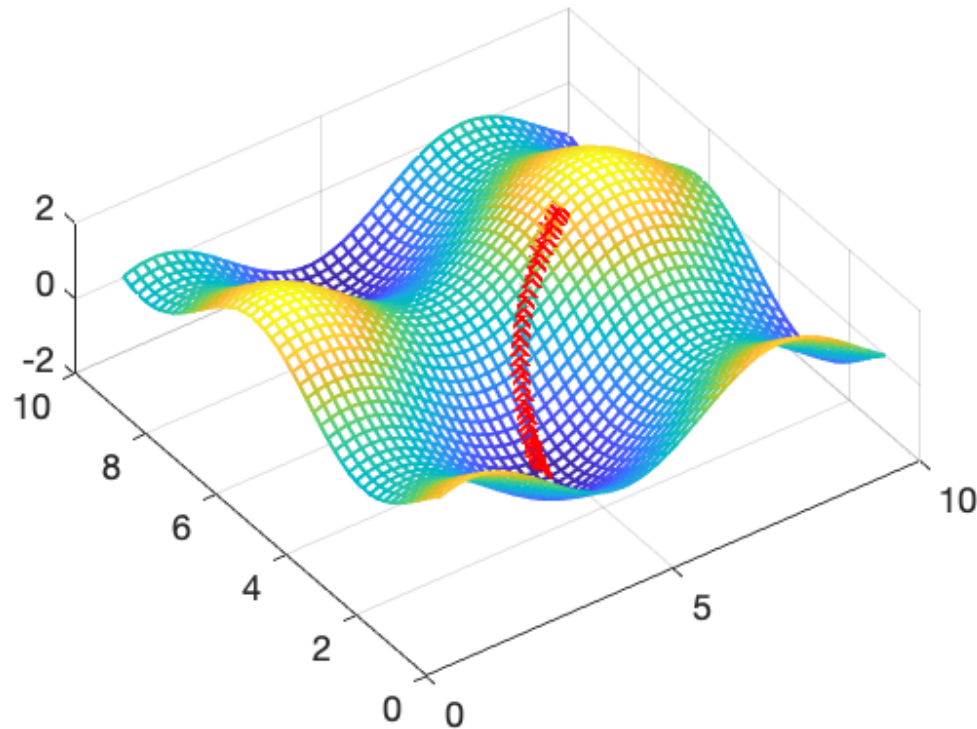
Gradient descent

- Algorithm:
 1. Initialize $\mathbf{w}_0$ (e.g., randomly)
 2. While not converged
 - 2.1. Update $\mathbf{w}_k \leftarrow \mathbf{w}_{k-1} - \eta \nabla_{\mathbf{w}} R(\mathbf{w}_{k-1})$
- The learning rate η has the same effect as in the 1D case
- Convergence can again be measured by
 - Thresholding the change in function value
 - Thresholding the change in parameters, e.g., $\|\mathbf{w}_{k-1} - \mathbf{w}_k\| < \delta_w$

Minimizing a non-convex function

- Example: Sine- and cosine-based function of 2 variables

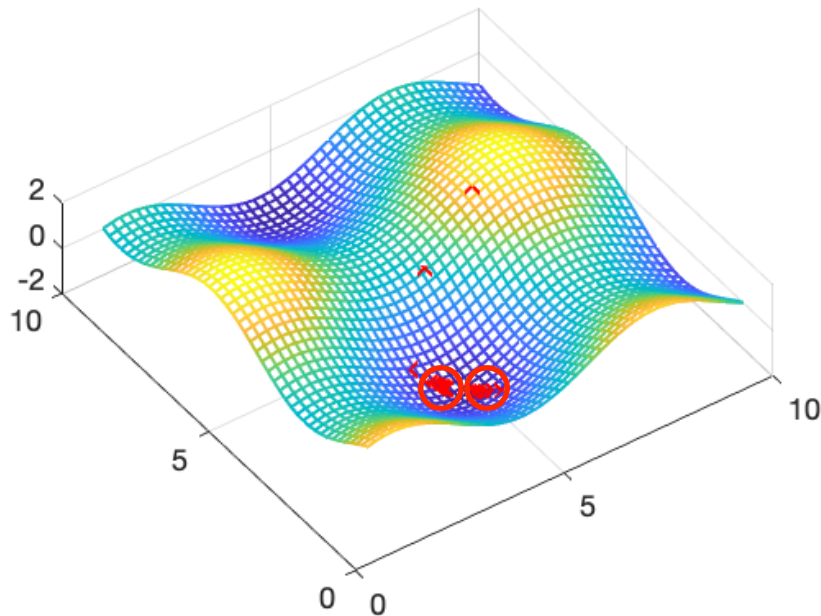
$$R(\mathbf{w}) = \sin(w^{(0)}) + \cos(w^{(1)}) \quad \nabla_{\mathbf{w}} R(\mathbf{w}) = \begin{bmatrix} \cos(w^{(0)}) \\ -\sin(w^{(1)}) \end{bmatrix}$$



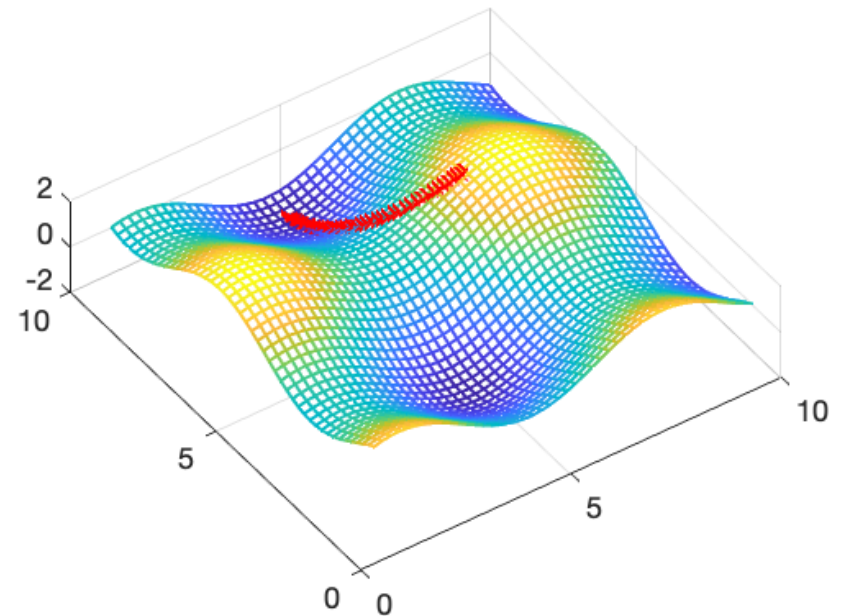
Gradient descent

- The same issues as in the 1D case can arise:
 - Different learning rates can yield different results
 - Different initializations can yield different results

$\eta = 2$ (vs 0.1)
Jumps between 2 solutions



$\mathbf{w}_0 = \begin{bmatrix} 7 \\ 6.5 \end{bmatrix}$ (vs $\begin{bmatrix} 7 \\ 6 \end{bmatrix}$)



End of the interlude

Back to Logistic Regression

Logistic regression: Training

- With binary ground-truth labels y_i , the cross-entropy can be written as

$$\begin{aligned} R(\mathbf{w}) &= - \sum_{i \in \text{positive samples}} \ln(\hat{y}_i(\mathbf{w})) - \sum_{i \in \text{negative samples}} \ln(1 - \hat{y}_i(\mathbf{w})) \\ &= - \sum_{i \in \text{positivesamples}} \ln \sigma(\mathbf{w}^T \mathbf{x}_i) - \sum_{i \in \text{negativesamples}} \ln(1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \end{aligned}$$

- To minimize it via gradient descent, we need to compute its gradient w.r.t. $\mathbf{w}$

Logistic regression: Gradient

- Using the chain rule, and after some derivations, we have

$$\nabla_{\mathbf{w}} \left(- \sum_{i \in \text{positive}} \ln \sigma(\mathbf{w}^T \mathbf{x}_i) \right) = \sum_{i \in \text{positive}} (\hat{y}_i - 1) \mathbf{x}_i = \sum_{i \in \text{positive}} (\hat{y}_i - y_i) \mathbf{x}_i$$

Because $y_i = 1$ for positive samples

- Similarly, for the negative examples, we have

$$\nabla_{\mathbf{w}} \left(- \sum_{i \in \text{negative}} \ln(1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \right) = \sum_{i \in \text{negative}} \hat{y}_i \mathbf{x}_i = \sum_{i \in \text{negative}} (\hat{y}_i - y_i) \mathbf{x}_i$$

Because $y_i = 0$ for negative samples

Logistic regression: Gradient

- Finally, this can be put back together to obtain

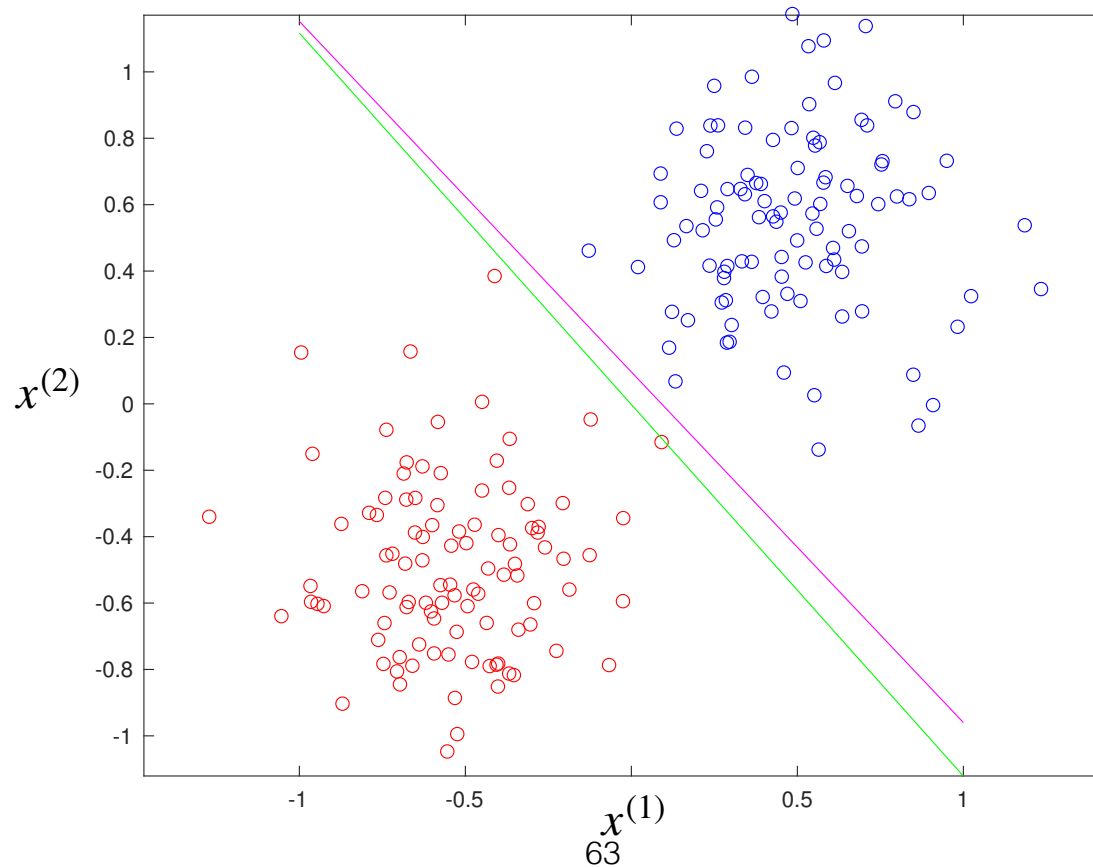
$$\nabla_{\mathbf{w}} R(\mathbf{w}) = \sum_{i=1}^N (\hat{y}_i - y_i) \mathbf{x}_i$$

where y_i is the true label for sample i , i.e., 1 for positive samples and 0 for negative samples

- Intuitively, the gradient is given by the error of the current prediction $(\hat{y}_i - y_i)$, multiplied by the input

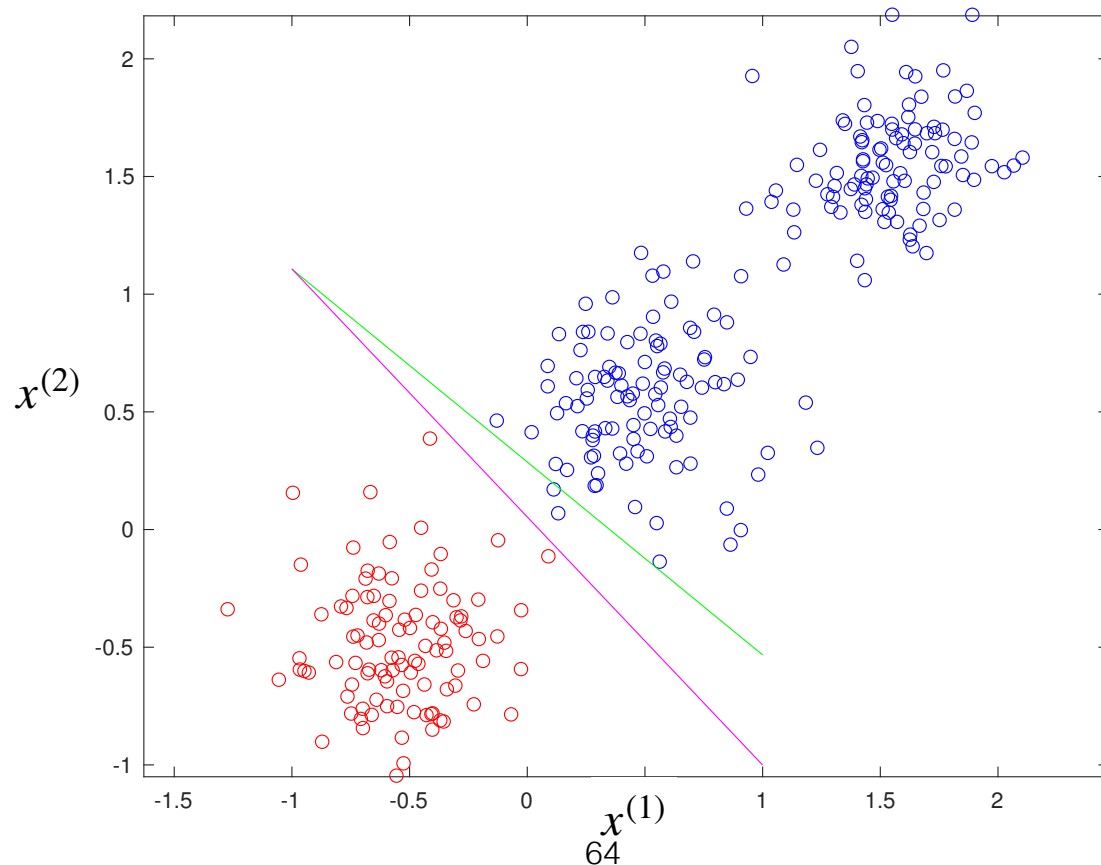
Logistic regression

- Example: 2D inputs, 2 classes
 - Least-square classification in green, logistic regression in magenta



Logistic regression

- Now, with additional samples, the boundary remains correct
 - Least-square classification in green, logistic regression in magenta



Logistic regression: Demo

- [https://playground.tensorflow.org/
#activation=sigmoid&batchSize=10&dataset=gauss®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=&seed=0.92907&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false](https://playground.tensorflow.org/#activation=sigmoid&batchSize=10&dataset=gauss®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=&seed=0.92907&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false)

Interpreting logistic regression

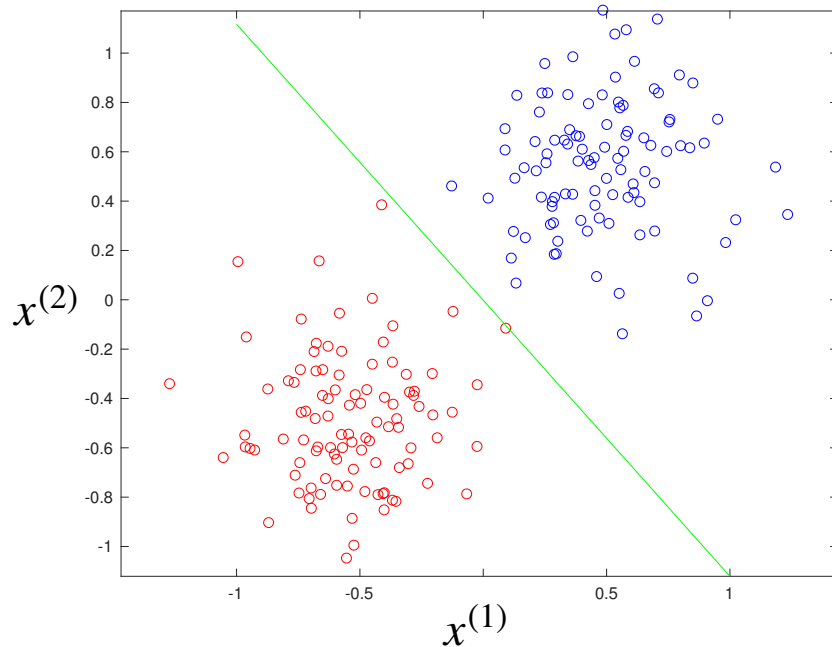
- As with linear regression, the learned coefficients of logistic regression can also be analyzed
- See a discussion of this at
 - <https://www.displayr.com/how-to-interpret-logistic-regression-coefficients/>

True/False

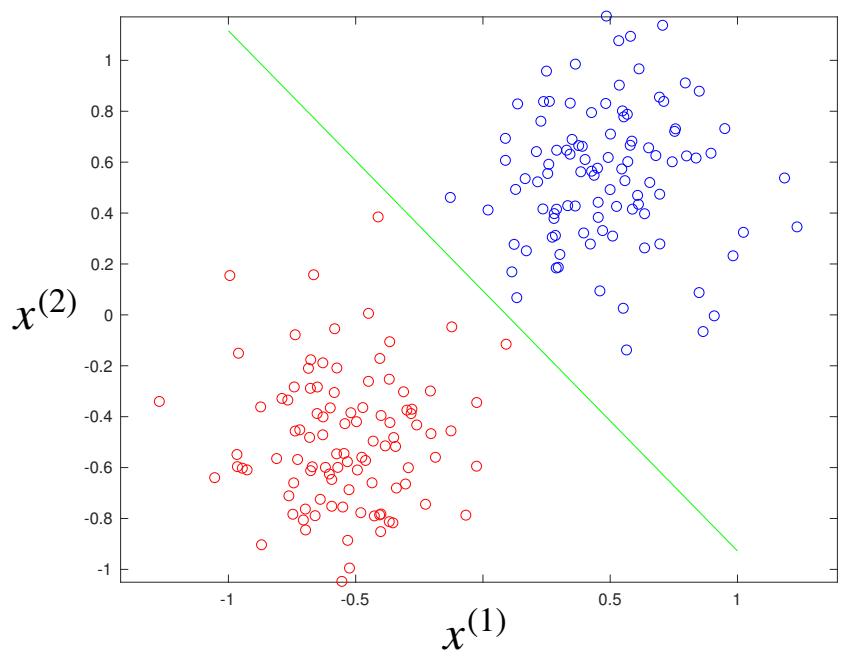
- Linear regression and logistic regression are both used to solve regression tasks where the output is a continuous value
- Linear regression and logistic regression are both linear ML models

Decision boundaries

- Different classifiers have different decision boundaries



Least-square classification

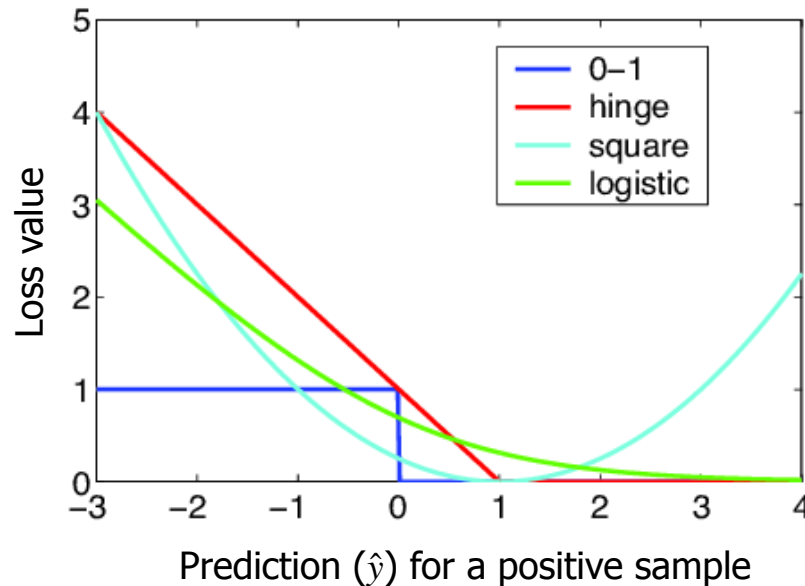


Logistic regression

- Yet they rely on the same linear model
 - So why do they differ?

Linear models

- The real difference between these methods is the loss function

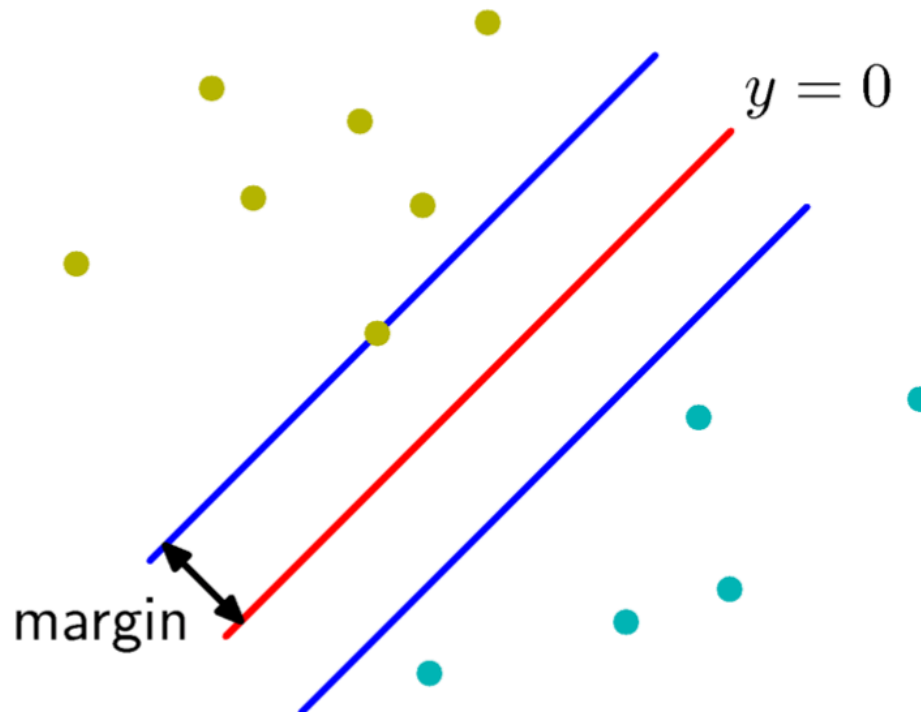


- Square: Least-square classification
- Logistic: Logistic regression
- Hinge: SVM
- 0-1: Ideal

- The choice of loss function has an impact on the resulting algorithm
- Let's have a look at the intuition behind SVM
 - This year I will spare you the mathematical details of SVM

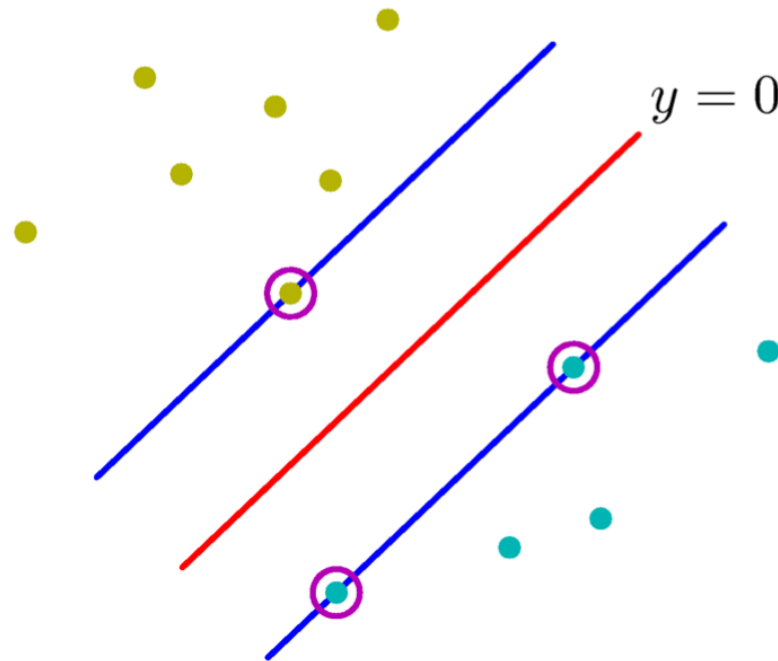
Margin (Chapter 7.1 in Bishop's book)

- The distance between the decision boundary and the nearest sample is called *margin*
 - Here, the decision boundary is taken at $\hat{y} = 0$, not 0.5 (labels are -1 and 1)



Maximizing the margin

- A good decision boundary should maximize the margin
 - Such a boundary is determined by a subset of the data points, named *support vectors* (indicated with circles)



Support Vector Machine

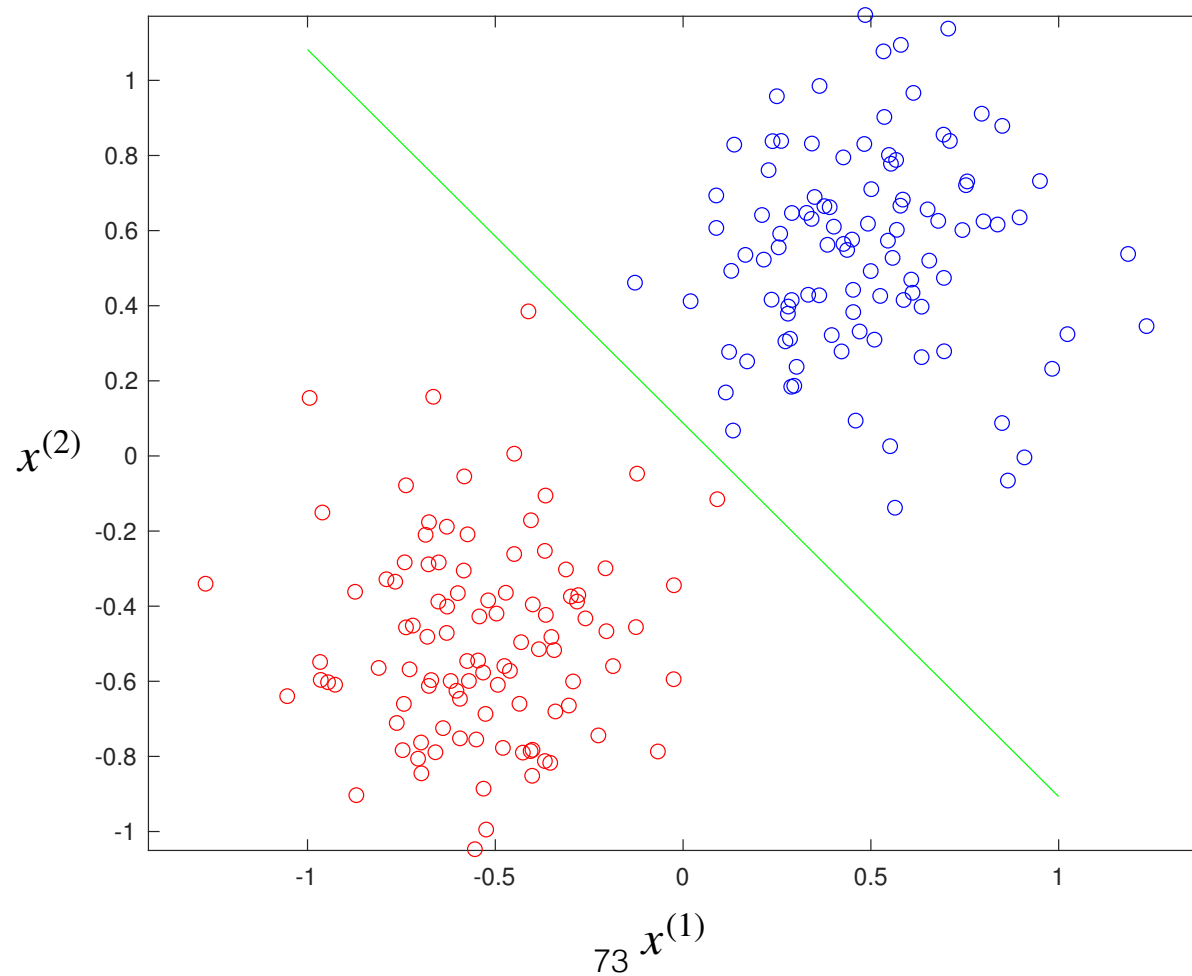
- The SVM classifier is a binary classifier based on the linear model as before:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

- It aims to maximize the margin while enforcing a correct classification for each training samples
- This is formulated as a convex optimization problem that can be solved to optimality with standard iterative solvers

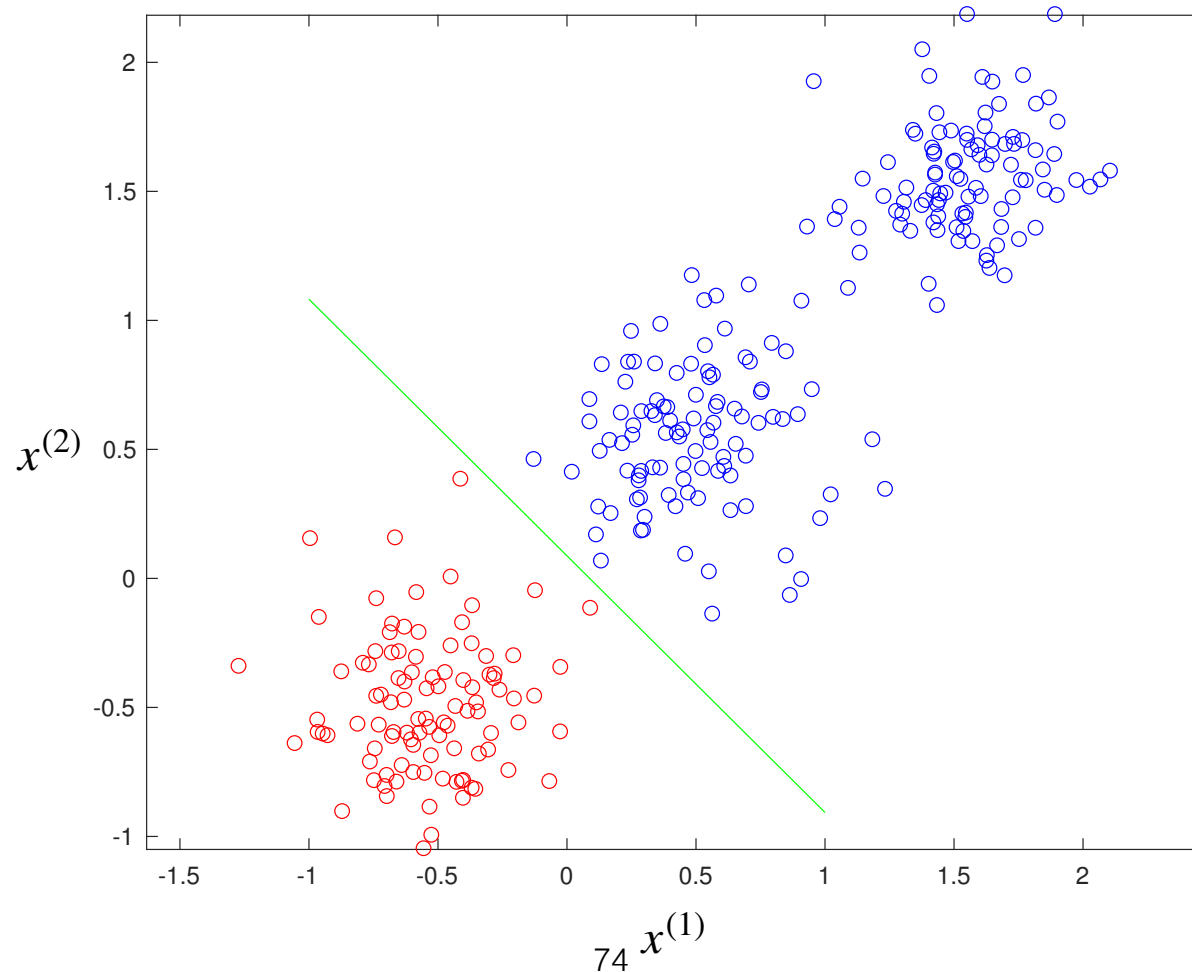
SVM Example

- Back to our toy data (2D samples from 2 classes)



SVM Example

- In the presence of additional samples, the decision boundary is unchanged



SVM: Loss function

- Ultimately, training an SVM can be written as

$$\min_{w^{(0)}, \tilde{\mathbf{w}}} \frac{1}{2C} \|\tilde{\mathbf{w}}\|^2 + \sum_{i=1}^N \max(0, 1 - y_i \cdot (w^{(0)} + \tilde{\mathbf{w}}^T \mathbf{x}_i))$$

- The term $\|\tilde{\mathbf{w}}\|^2$ corresponds to the inverse of the margin (distance between the decision boundary and the nearest sample); minimizing it seeks to maximize the margin

- The term

$$\max(0, 1 - y_i \cdot (w^{(0)} + \tilde{\mathbf{w}}^T \mathbf{x}_i))$$

is referred to as the *hinge loss* and encourages correctly classifying each sample

Support vector machine: Demo

- <https://jgreitemann.github.io/svm-demo>